NUS-ISS
Intelligent Sensing and Sense Making

Module 3 - Workshop on sensor signal processing

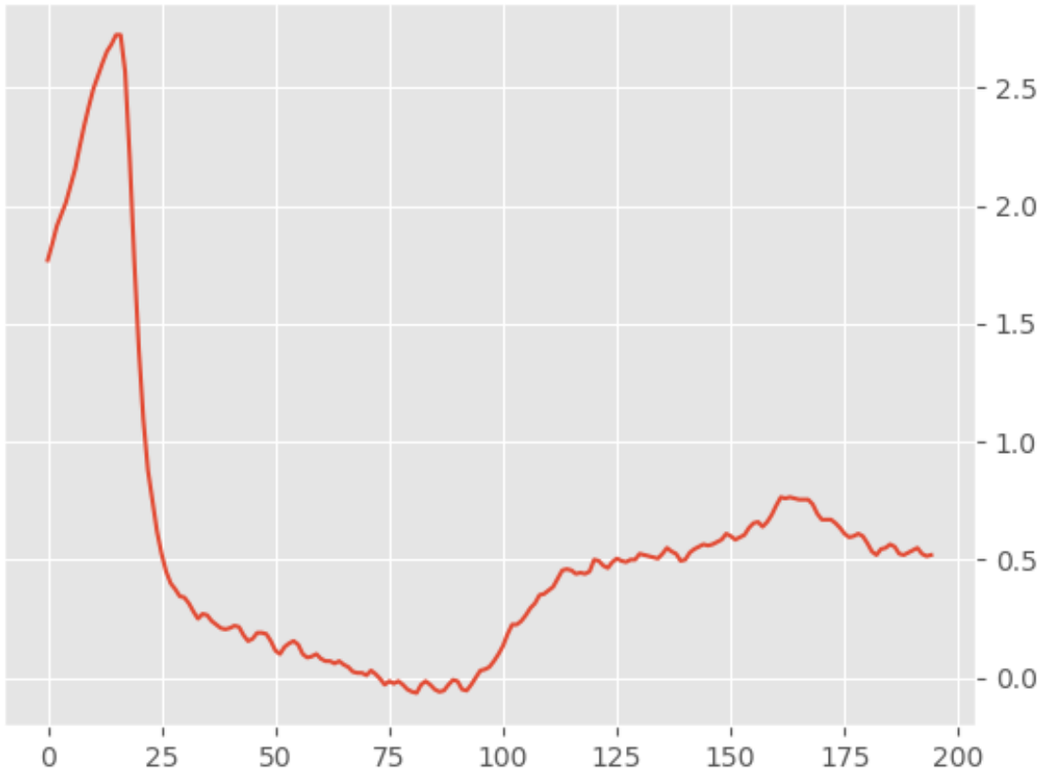
by Hu Zhengqing

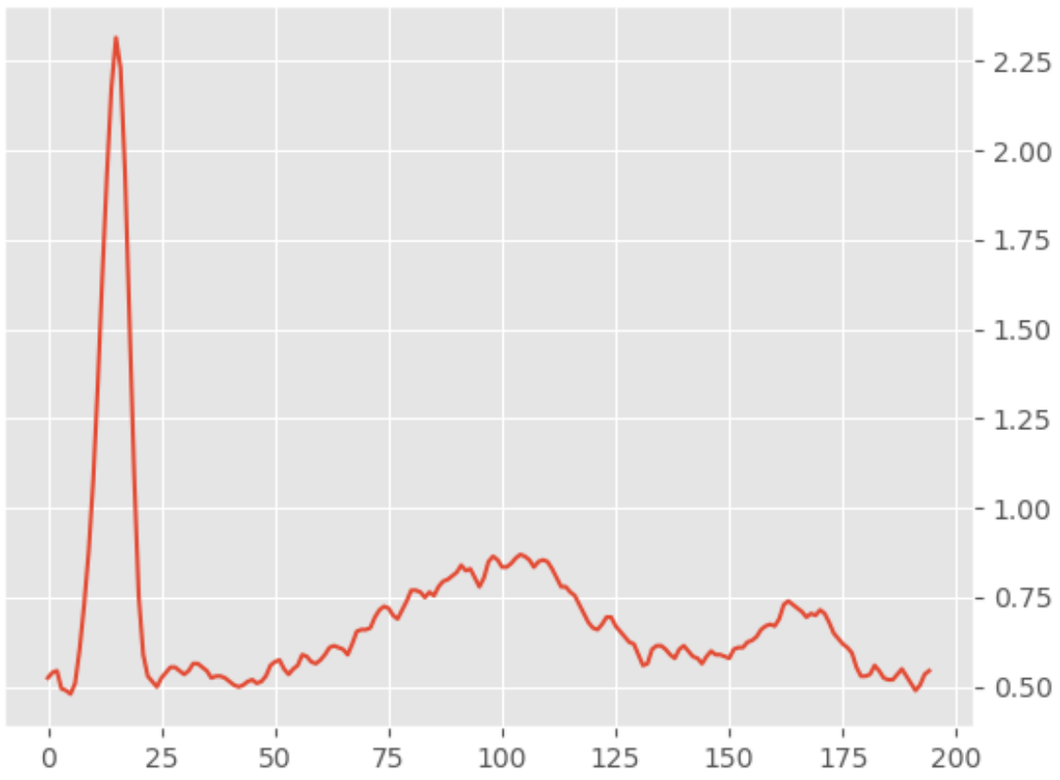
© 2024 National University of Singapore.
All Rights Reserved.

Dynamic Time Warping

Problem

- How similar are these two signals?
- In which manners are they similar?





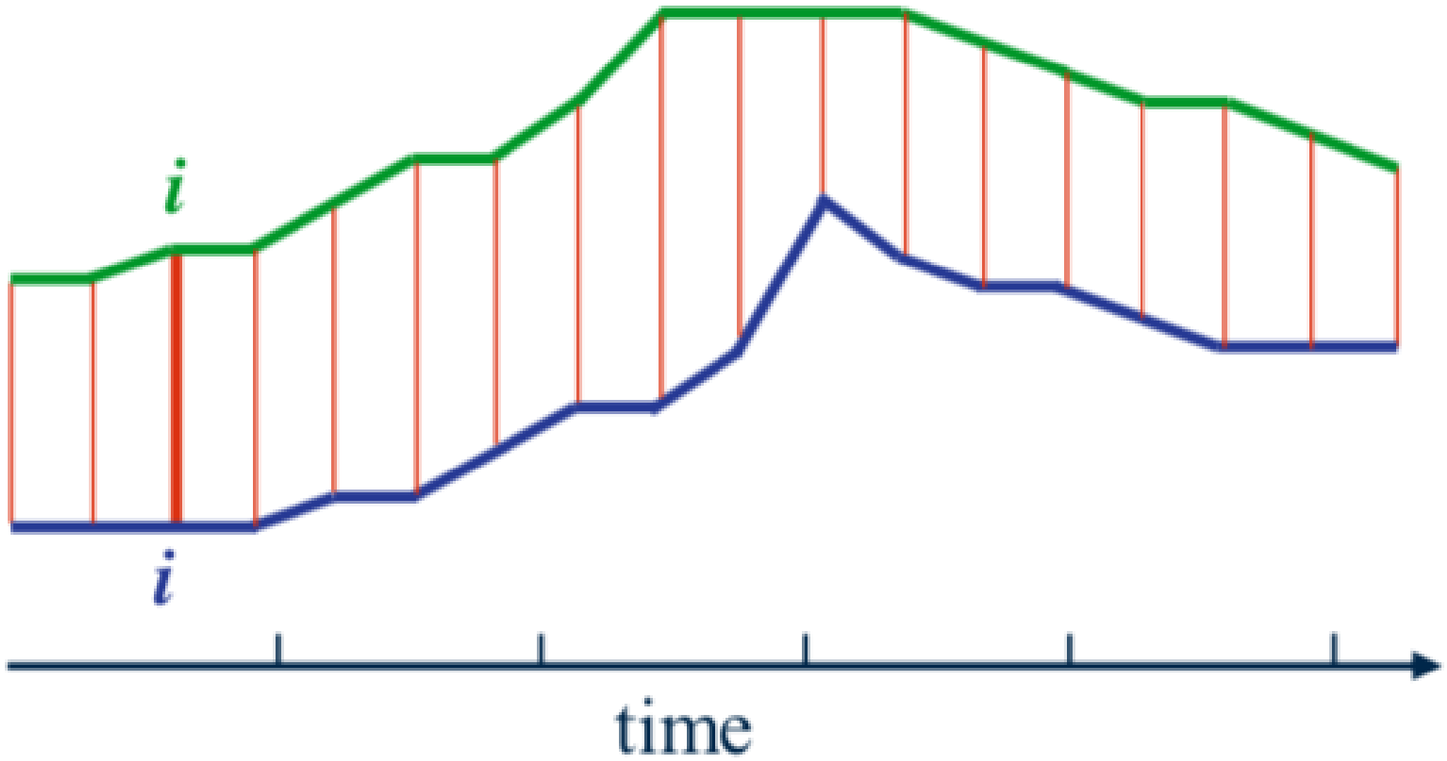
© 2024 National University of Singapore. All Rights Reserved.

Can we try ...

Euclidean, Manhattan .....?

- We can measure the similarity of two signals by calculating the distance between the i-th point on one signal and the i-th point on another signal

- Simple concept, but could not capture the similarity in shape



Source: "Dynamic Time Warping Algorithm", by Elena Tsiporkova

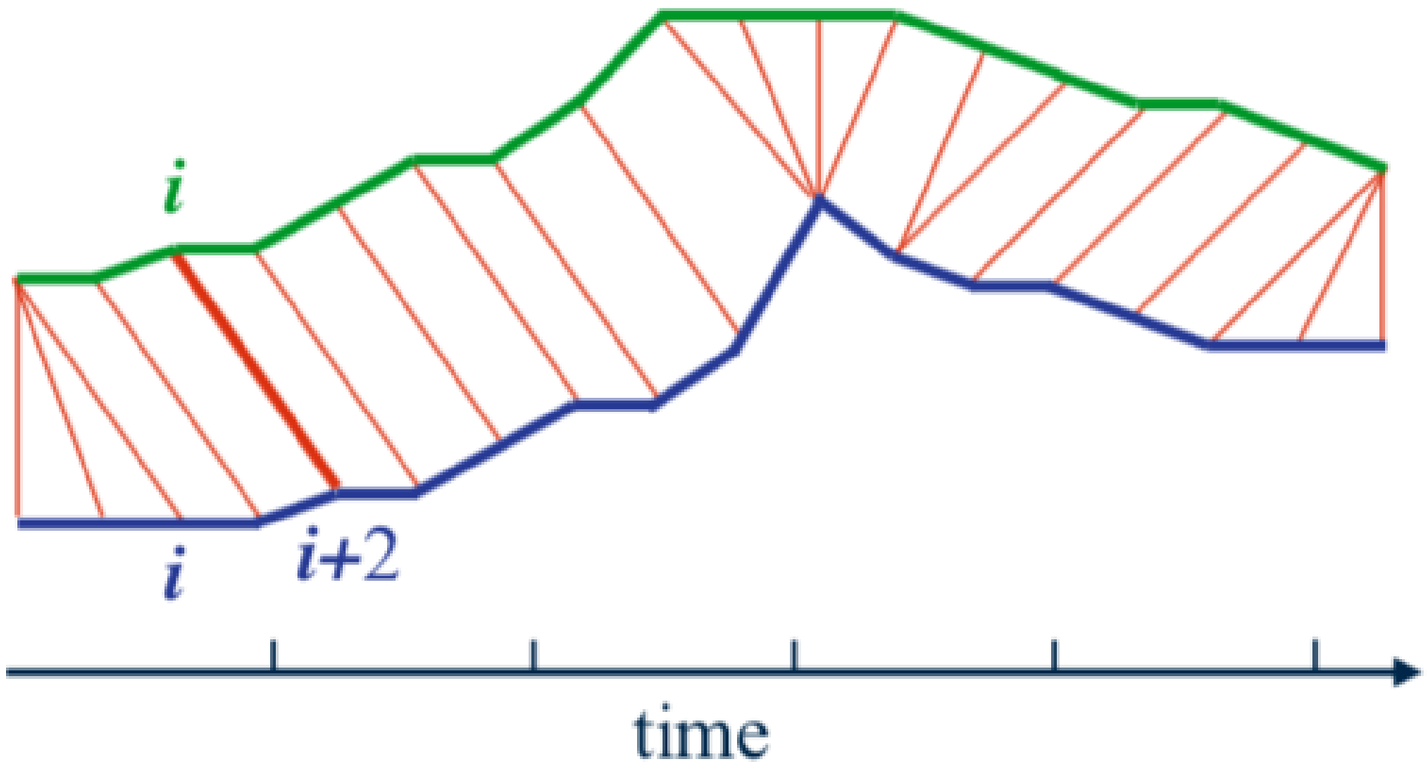
© 2024 National University of Singapore. All Rights Reserved.

How about ...

non-linear alignment .....?

- Elastic alignment between points of two signals produces a better, more intuitive similarity measure

- Allow similar shapes to match even if they are out of phase



Source: "Dynamic Time Warping Algorithm", by Elena Tsiporkova

© 2024 National University of Singapore. All Rights Reserved.

Distance

Another term to say 'similiarity'

- Consider two distinct signals

$$\mathbf{x} = [x_1, x_2, \ldots, x_i, \ldots x_m]$$
$$\mathbf{y} = [y_1, y_2, \ldots, y_j, \ldots y_n]$$

- The distance between the two signals is defined as

$$d_s(\mathbf{x}, \mathbf{y})$$

- Euclidean distance between two signals:

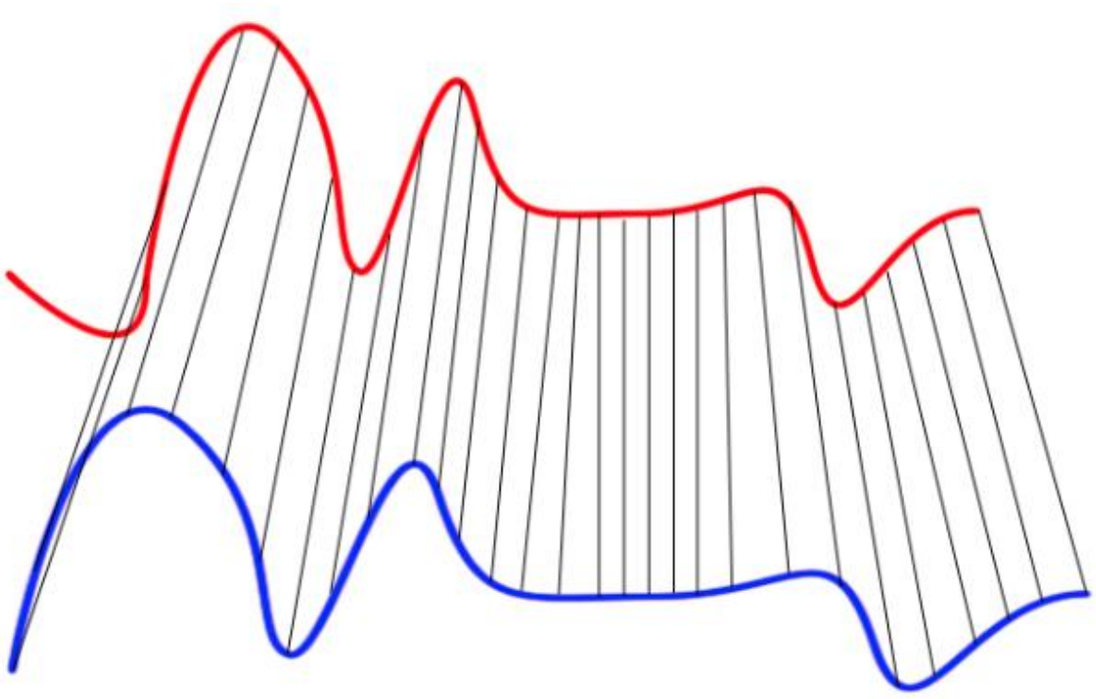
$$d_s(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (x_i - y_i)^2}$$

- **Problem: two signals must be of same length!**

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

DTW



Source: https://th.wikipedia.org/wiki/Dynamic_time_warping#/media/File:Euclidean_vs_DTW.jpg

- An algorithm to measure similarity between two temporal sequences (signal), which may vary in speed
- DTW calculates an optimal match between two given sequences
- Sequences are warped along time dimension to determine similarity independent of variations in time
- DTW produces warping path, which enables alignment between two signals

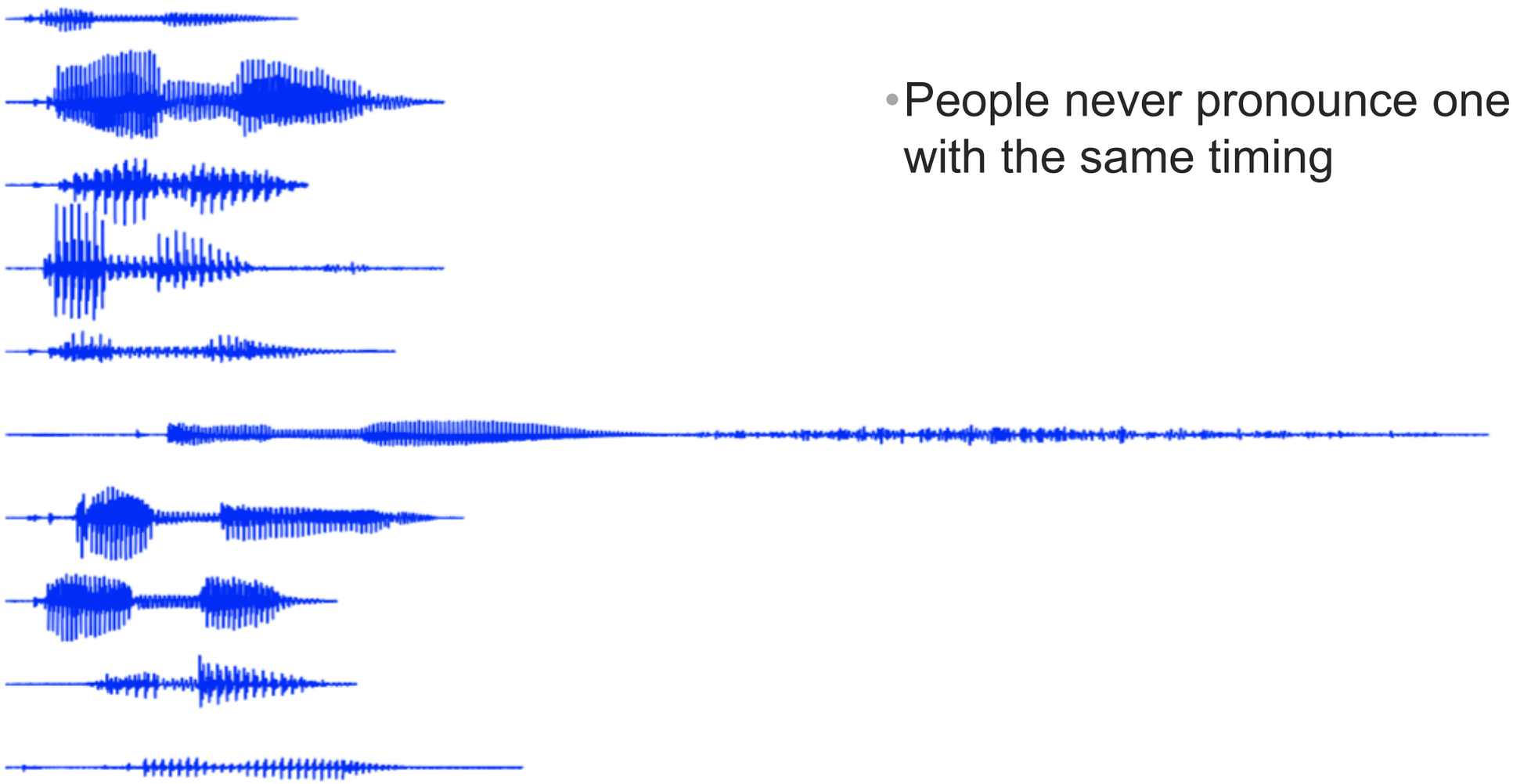
© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Usage

- Commonly used in speech recognition
- Individual never pronounces one word twice in exact way
- People never pronounce one word with the same timing

"timing"

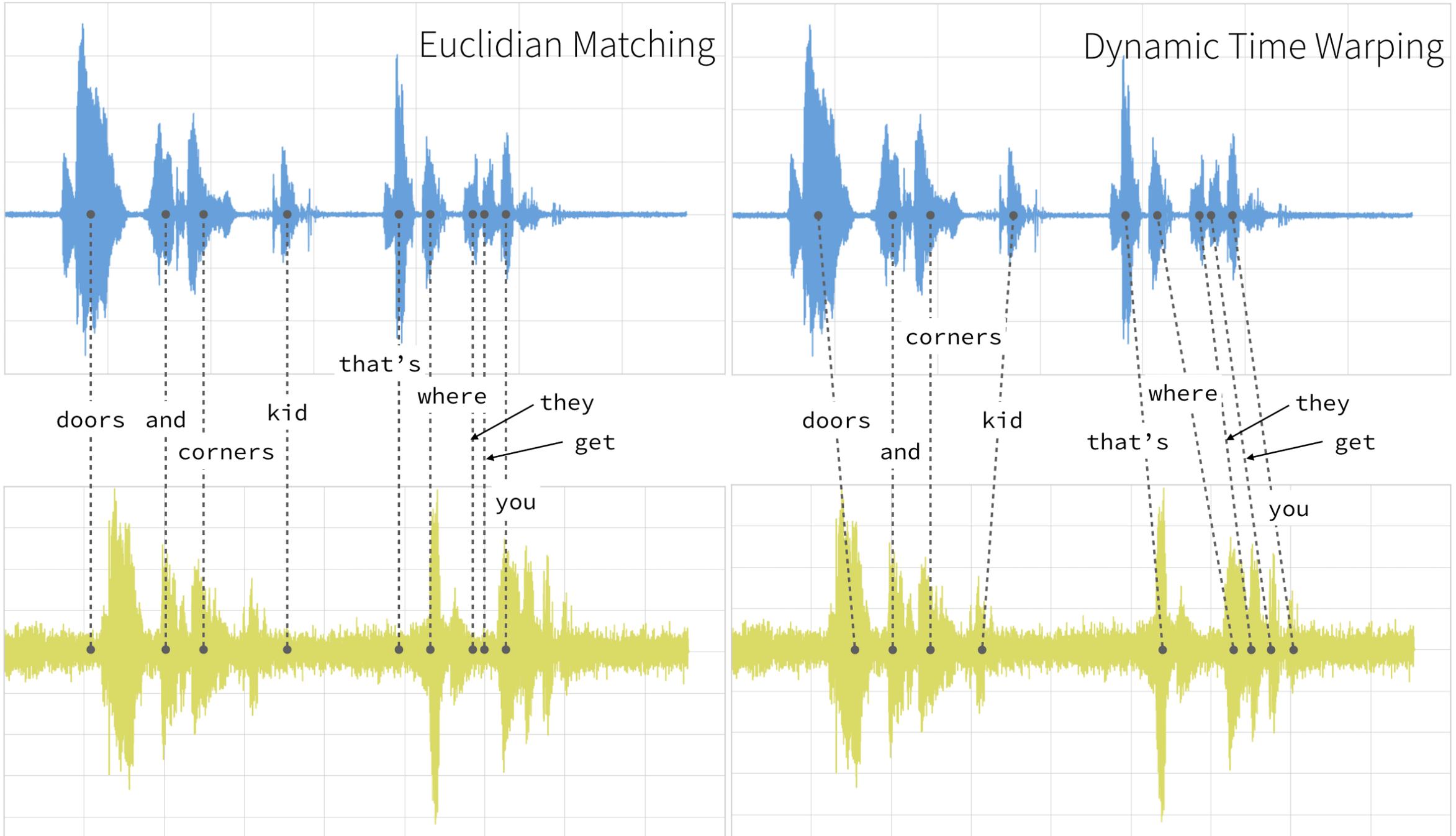


Source: "Speech Recognition - Intro and DTW", by Jan Černocky

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Vs Euclidean (Comparison of Audio Clips Example)



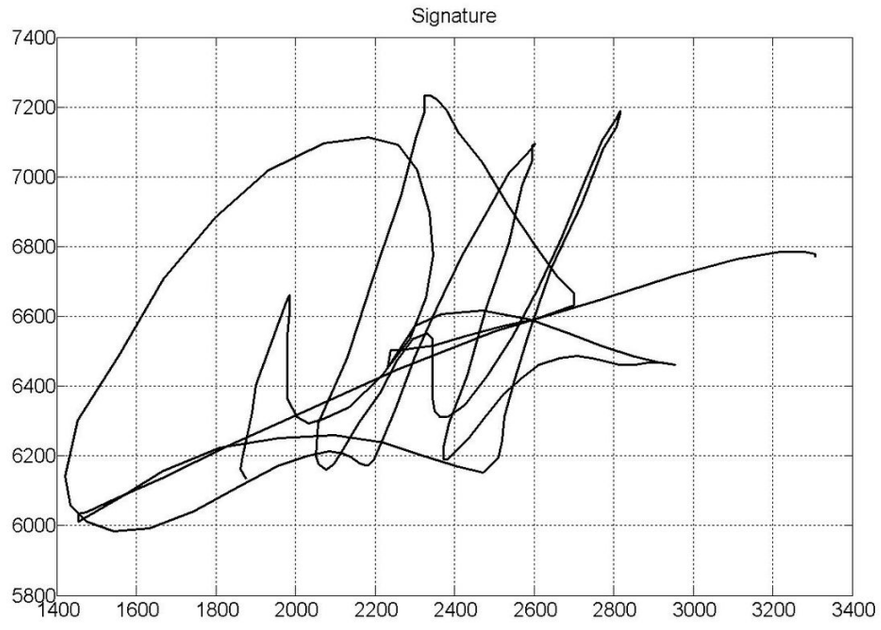
© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

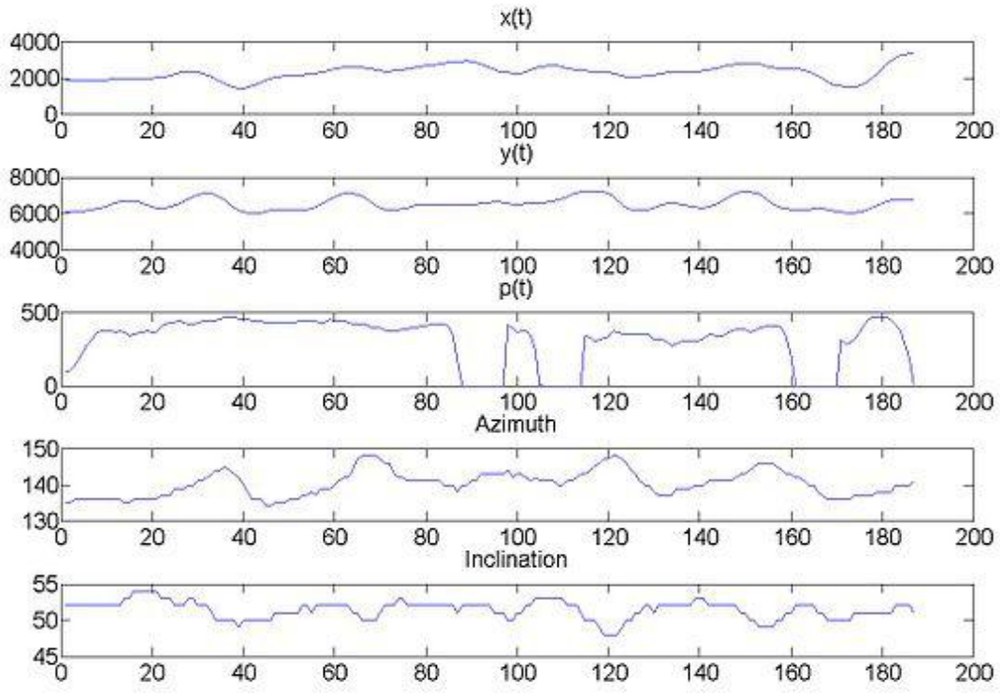
Dynamic signature recognition

- Users sign their signature on digital tablet
- Dynamic information captured:
  - x position
  - y position
  - pressure
  - azimuth
  - inclination

- Use DTW to check / match signature



Source: "Speech Recognition - Intro and DTW", by Jan Černocky



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Overview of algorithm

n x m matrix covers the window boundaries

orange curve = correct wrapping path

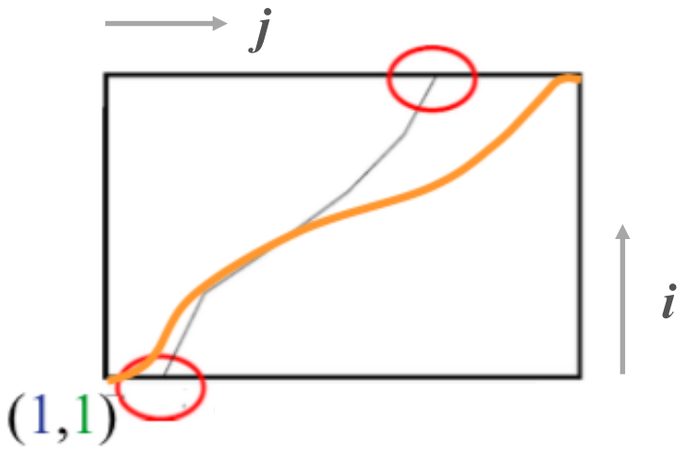
- Start by constructing *n* (row) x *m* (clm) matrix *D*, in which

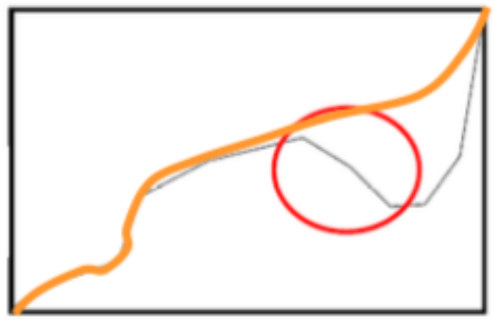
$$D_{i,j} = d_s(y_i, x_j)$$

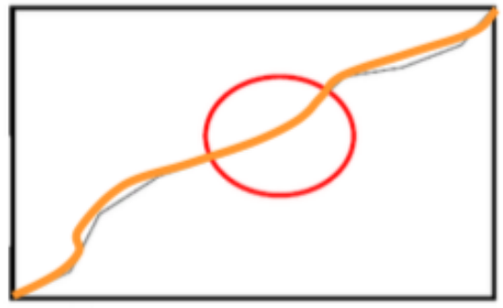
- where

$$d_s(y_i, x_j) = (y_i - x_j)^2$$

- Create a warping path *w* that maps points between x and y, the path w must satisfy the following rules:
  1. Boundary conditions
  2. Monotonicity
  3. Continuity







Source: "Dynamic time warping algorithm", by Elena Tsiporkova

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Overview of algorithm

1st Rule: start point from bottom left and end point at top right

2nd Rule: Cannot move back in direction

3rd Rule: cannot have a break in between! Must be continuous

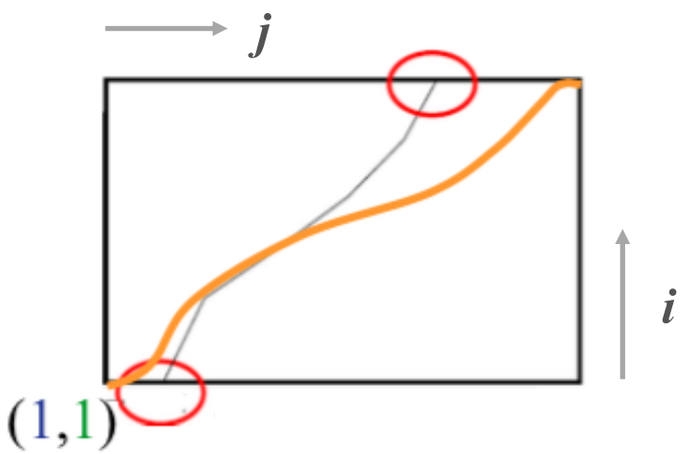
- Start by constructing n (row) x m (clm) matrix D, in which

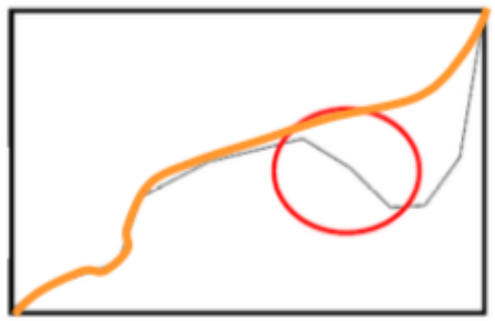
$$D_{i,j} = d_s(y_i, x_j)$$

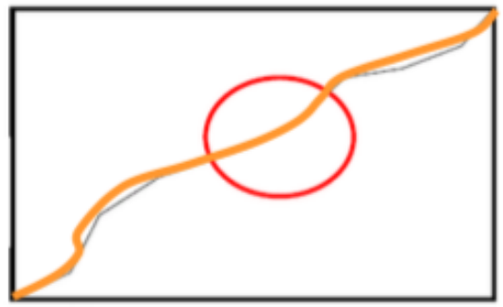
- where

$$d_s(y_i, x_j) = (y_i - x_j)^2$$

- Create a warping path w that maps points between x and y, the path w must satisfy the following rules:
  1. Boundary conditions
  2. Monotonicity
  3. Continuity







Source: "Dynamic time warping algorithm", by Elena Tsiporkova

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Overview of algorithm

- DTW algorithm consists of mainly 3 parts:
  1. Compute distance matrix
  2. Compute accumulated cost matrix
  3. Search the optimal path

- To start the code, import the necessary libraries, and setup a bit

Codes to import libraries and to perform some setup

```
> import numpy as np
> import matplotlib.pyplot as plt
> import pandas as pd

> plt.style.use('ggplot')
> plt.rcParams['ytick.right']      = True
> plt.rcParams['ytick.labelright'] = True
> plt.rcParams['ytick.left']       = False
> plt.rcParams['ytick.labelleft'] = False
```


© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

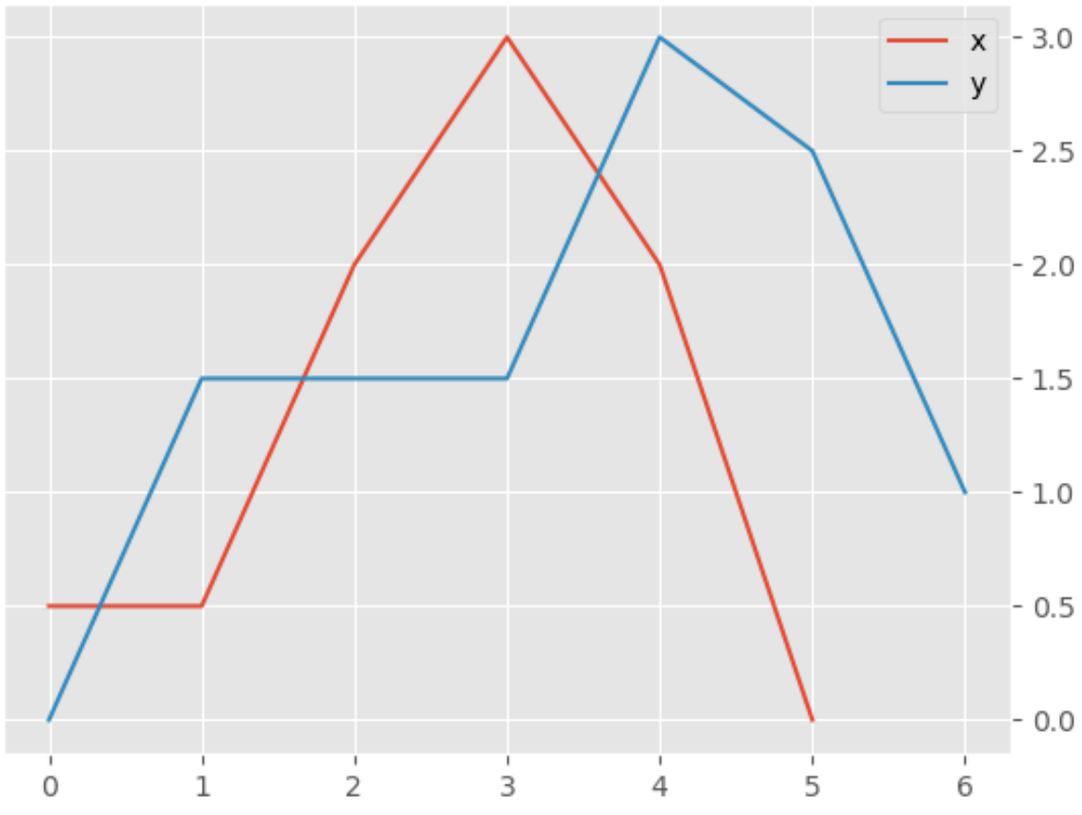
1. Compute distance matrix

- Define two simple short signals:

```
> x = np.array([0.5,0.5,2.0,3.0,2.0,0.0])
> y = np.array([0.0,1.5,1.5,1.5,3.0,2.5,1.0])
```

- Plot the two signals

```
> plt.figure()
> plt.plot(x,
           color="C0",
           label='x')
> plt.plot(y,
           color="C1",
           label='y')
> plt.legend()
```



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

1. Compute distance matrix

applying maths formula to calculate the distance matrix in DTW

- Compute the distance matrix is straightforward, since the matrix is defined as

$$D_{i,j} = d_s(y_i, x_j)$$

- and

$$d_s(y_i, x_j) = (y_i - x_j)^2$$

- The corresponding code

```
> dists = np.zeros((len(y),len(x)))

> for i in range(len(y)):
      for j in range(len(x)):
          dists[i,j]  = (y[i]-x[j])**2
```

dists

```
[[0.25, 0.25, 4.  , 9.  , 4.  , 0.  ],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [6.25, 6.25, 1.  , 0.  , 1.  , 9.  ],
 [4.  , 4.  , 0.25, 0.25, 0.25, 6.25],
 [0.25, 0.25, 1.  , 4.  , 1.  , 1.  ]]
```


© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

1. Compute distance matrix

Note: Output from these codes is inverted

i.e. top left cell = D[0,0]; bottom left should be D[0,0] instead

- Compute the distance matrix is straightforward, since the matrix is defined as

$$D_{i,j} = d_s(y_i, x_j)$$

- and

$$d_s(y_i, x_j) = (y_i - x_j)^2$$

- The corresponding code

```
> dists = np.zeros((len(y),len(x)))

> for i in range(len(y)):
      for j in range(len(x)):
          dists[i,j]  = (y[i]-x[j])**2
```

dists

```
[[0.25, 0.25, 4.  , 9.  , 4.  , 0.  ],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [6.25, 6.25, 1.  , 0.  , 1.  , 9.  ],
 [4.  , 4.  , 0.25, 0.25, 0.25, 6.25],
 [0.25, 0.25, 1.  , 4.  , 1.  , 1.  ]]
```


© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

1. Compute distance matrix

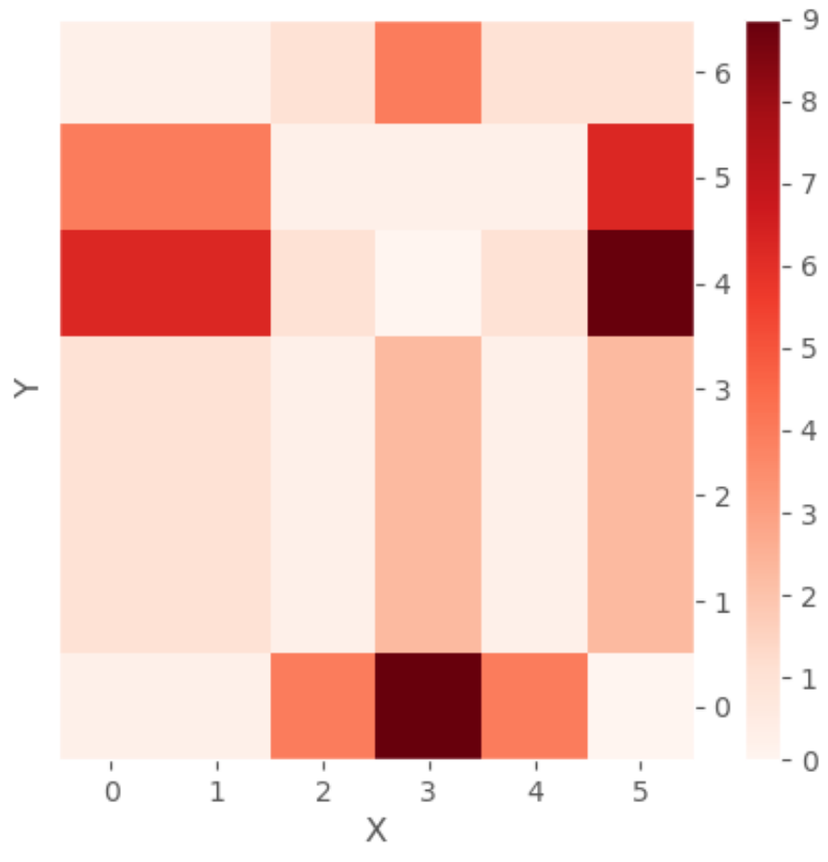
- Create a function to do a plot on the distance matrix

```
> def pltDistances(dists,xlab="X",ylab="Y",clrmap="viridis"):
      imgplt  = plt.figure()
      plt.imshow(dists,
                 interpolation='nearest',
                 cmap=clrmap)

      plt.gca().invert_yaxis()
      plt.xlabel(xlab)
      plt.ylabel(ylab)
      plt.grid()
      plt.colorbar()

      return imgplt
```

Inverted because viridis starts from top left




```
> pltDistances(dists,clrmap='Reds')
```

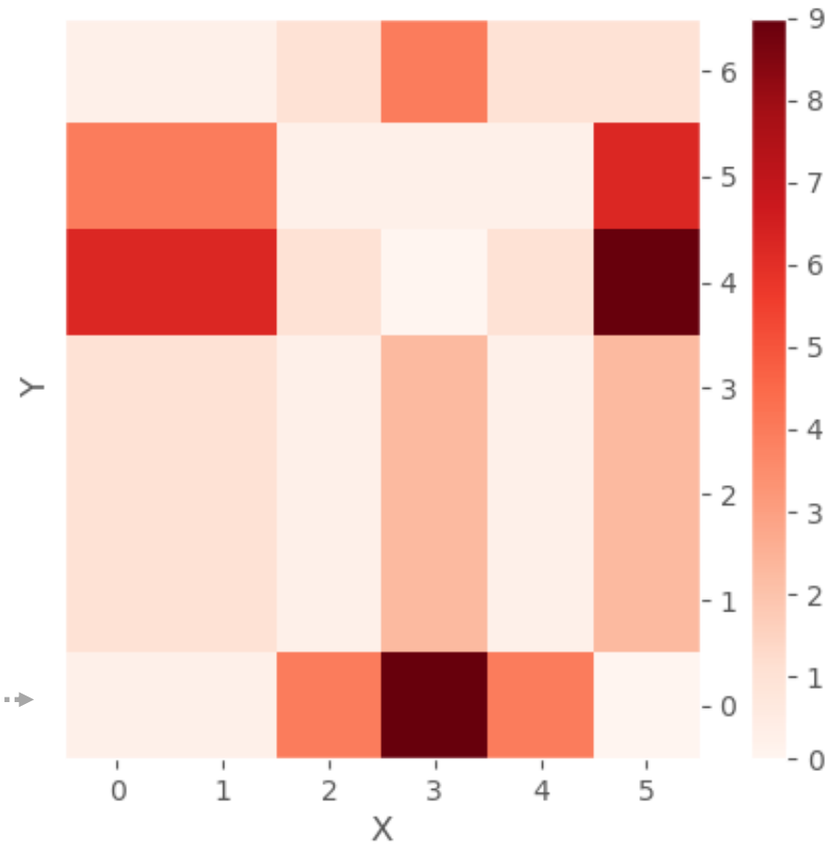

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

1. Compute distance matrix

- Do take note, in the plot, the y axis is inverted
- Thus, first row of the matrix corresponds to the last row in the figure

```
[[0.25, 0.25, 4.  , 9.  , 4.  , 0.  ],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [1.  , 1.  , 0.25, 2.25, 0.25, 2.25],
 [6.25, 6.25, 1.  , 0.  , 1.  , 9.  ],
 [4.  , 4.  , 0.25, 0.25, 0.25, 6.25],
 [0.25, 0.25, 1.  , 4.  , 1.  , 1.  ]]
```



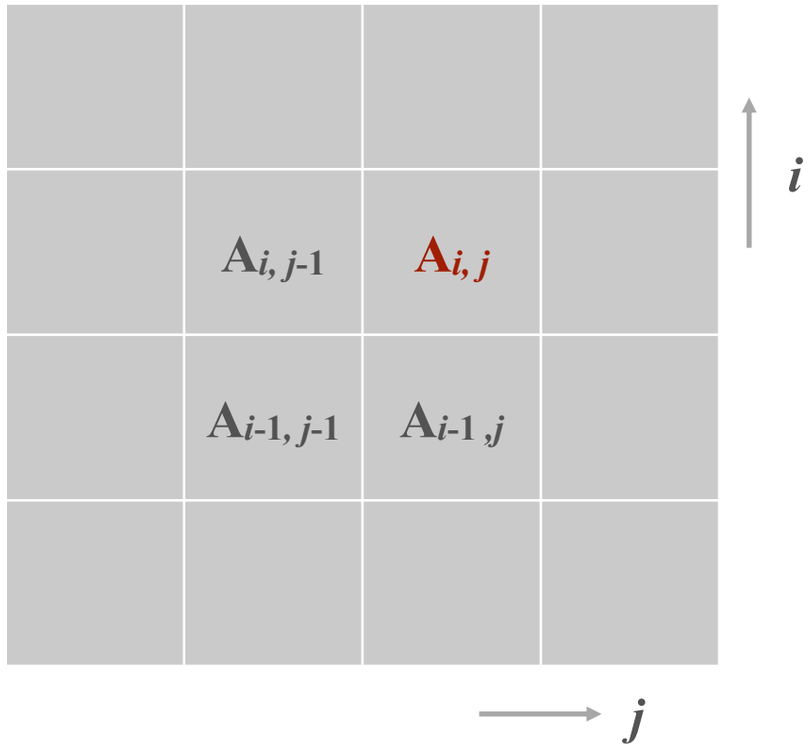
© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

2. Compute accumulated cost matrix

Cost matrix important to determine wrapping path!

$\mathbf{A}_{i,j}$ equals to $\mathbf{D}_{i,j}$ plus either $\mathbf{A}_{i-1,j-1}$, $\mathbf{A}_{i,j-1}$ or $\mathbf{A}_{i-1,j}$, whichever has the lowest value



- The accumulated cost matrix is defined

$$A_{i,j} = D_{i,j} + min(A_{i-1,j}, A_{i,j-1}, A_{i-1,j-1})$$

- When *i* and *j* equals to 0

$$A_{0,0} = D_{0,0}$$

- When *i* equals to 0 (first row)

$$A_{0,j} = D_{0,j} + A_{0,j-1}$$

- When *j* equals to 0 (first column)

$$A_{i,0} = D_{i,0} + A_{i-1,0}$$

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

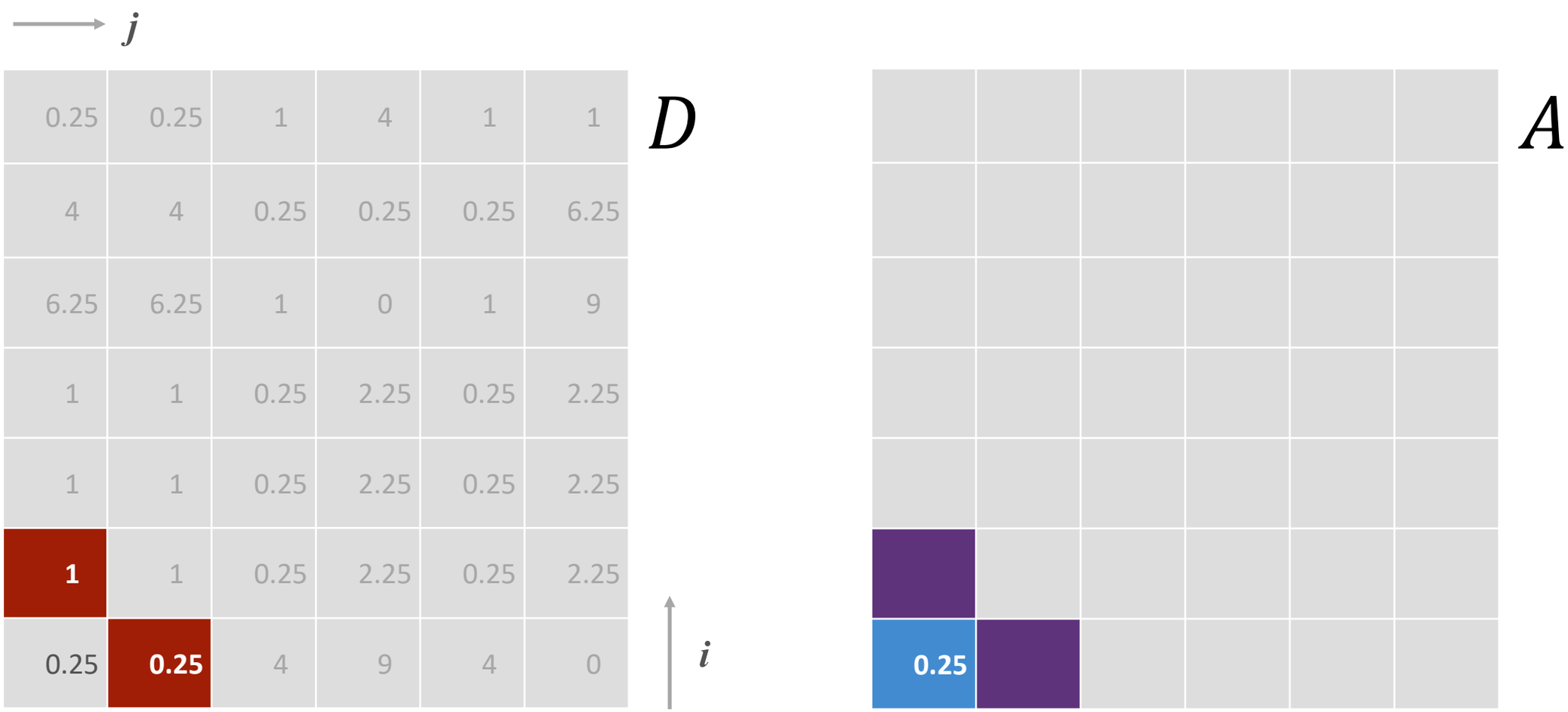
2. Compute accumulated cost matrix

- When *i* equals to 0 (first row)

$$A_{0,j} = D_{0,j} + A_{0,j-1}$$

- When *j* equals to 0 (first column)

$$A_{i,0} = D_{i,0} + A_{i-1,0}$$



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

2. Compute accumulated cost matrix

- When *i* equals to 0 (first row)

$$A_{0,j} = D_{0,j} + A_{0,j-1}$$

- When *j* equals to 0 (first column)

$$A_{i,0} = D_{i,0} + A_{i-1,0}$$

j →

0.25	0.25	1	4	1	1
4	4	0.25	0.25	0.25	6.25
6.25	6.25	1	0	1	9
1	1	0.25	2.25	0.25	2.25
1	1	0.25	2.25	0.25	2.25
1	1	0.25	2.25	0.25	2.25
0.25	0.25	4	9	**4**	0

D

i ↑

9.5					
3.25					
2.25					
1.25					
0.25	0.5	4.5	**13.5**		

A

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

2. Compute accumulated cost matrix

- Else

$$A_{i,j} = D_{i,j} + min(A_{i-1,j}, A_{i,j-1}, A_{i-1,j-1})$$

$j \longrightarrow$

0.25	0.25	1	4	1	1	D
4	4	0.25	0.25	0.25	6.25	
6.25	6.25	1	0	1	9	
1	1	0.25	2.25	0.25	2.25	
1	1	0.25	2.25	0.25	2.25	
1	**1**	0.25	2.25	0.25	2.25	
0.25	0.25	4	9	4	0	

$i \uparrow$

13.75						A
13.5						
9.5						
3.25						
2.25						
1.25						
0.25	0.5	4.5	13.5	17.5	17.5	

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

2. Compute accumulated cost matrix

- Else

$$A_{i,j} = D_{i,j} + min(A_{i-1,j}, A_{i,j-1}, A_{i-1,j-1})$$

$j \longrightarrow$

0.25	0.25	1	4	1	1
4	4	0.25	0.25	0.25	6.25
6.25	6.25	1	0	1	**9**
1	1	0.25	2.25	0.25	2.25
1	1	0.25	2.25	0.25	2.25
1	1	0.25	2.25	0.25	2.25
0.25	0.25	4	9	4	0

D

$i \uparrow$

13.75					
13.5					
9.5	9.5	2.25	1.25	**2.25**	
3.25	3.25	1.25	3.25	3.25	5.5
2.25	2.25	1	3	3.25	5.5
1.25	1.25	0.75	3	3.25	5.5
0.25	0.5	4.5	13.5	17.5	17.5

A

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

2. Compute accumulated cost matrix

- Name the accumulated cost matrix as `acuCost`. It has the same shape as `dists`.

Codes to determine the accumulated cost matrix

$A_{0,0} = D_{0,0}$

```
> acuCost     = np.zeros(dists.shape)
> acuCost[0,0]= dists[0,0]
```

starts from 1 because acuCost[0,0] has been filled with dists[0,0]

$A_{0,j} = D_{0,j} + A_{0,j-1}$

first row

```
> for j in range(1,dists.shape[1]):
      acuCost[0,j]    = dists[0,j]+acuCost[0,j-1]
```

$A_{i,0} = D_{i,0} + A_{i-1,0}$

first column

```
> for i in range(1,dists.shape[0]):
      acuCost[i,0]    = dists[i,0]+acuCost[i-1,0]
```

$A_{i,j} = D_{i,j} + min(A_{i-1,j}, A_{i,j-1}, A_{i-1,j-1})$

```
> for i in range(1,dists.shape[0]):
      for j in range(1,dists.shape[1]):
          acuCost[i,j]    = min(acuCost[i-1,j-1],
                                acuCost[i-1,j],
                                acuCost[i,j-1])+dists[i,j]
```

```
> pltDistances(acuCost,clrmap='Reds')
```

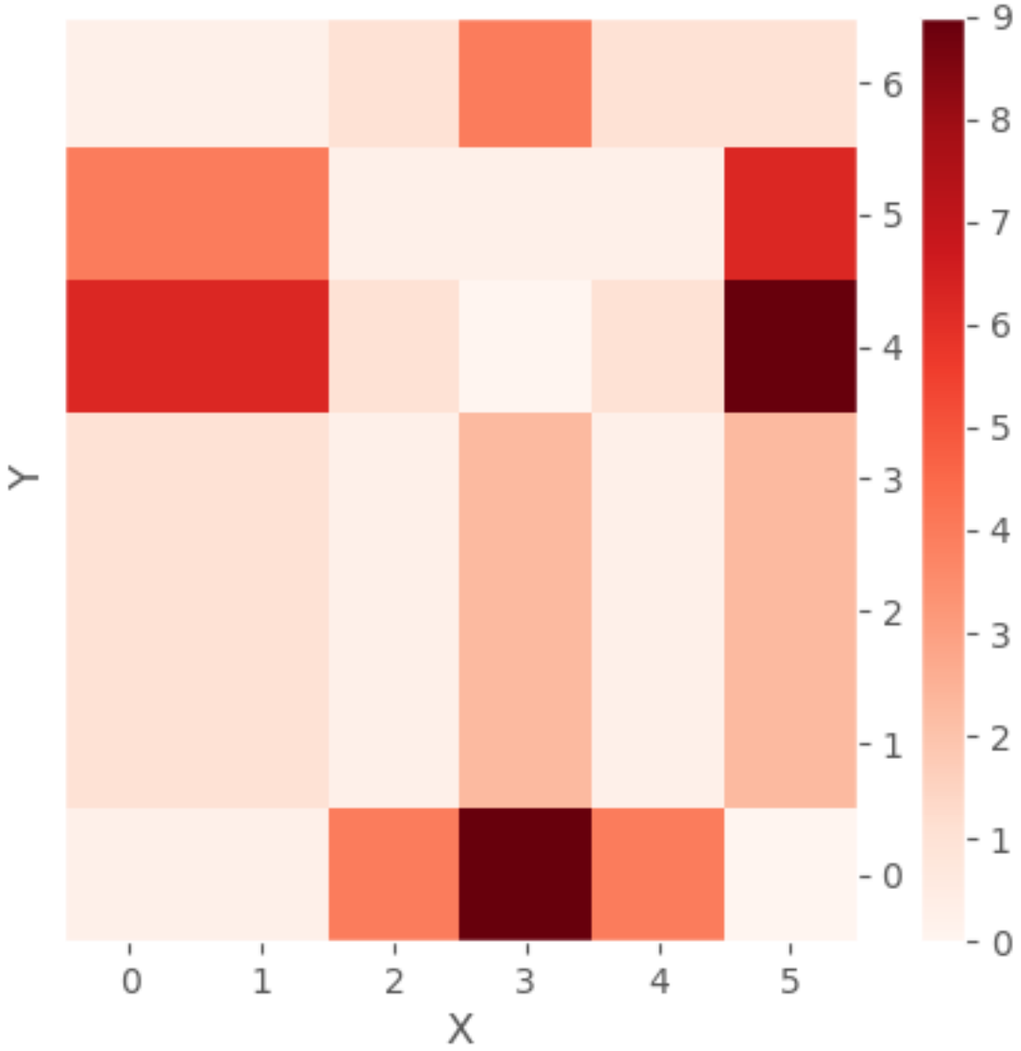

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

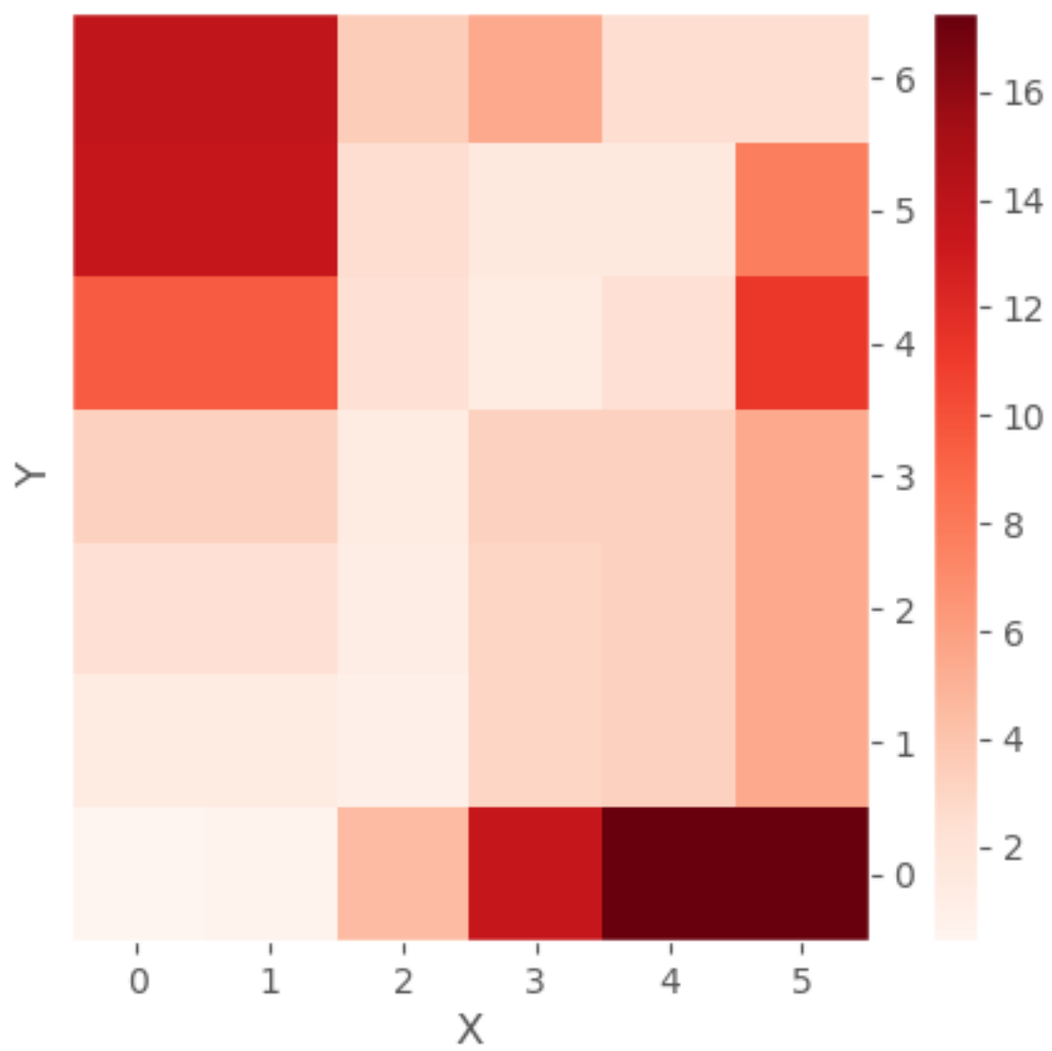
2. Compute accumulated cost matrix

- Can you see the warping / optimal path?

dists



acuCost



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

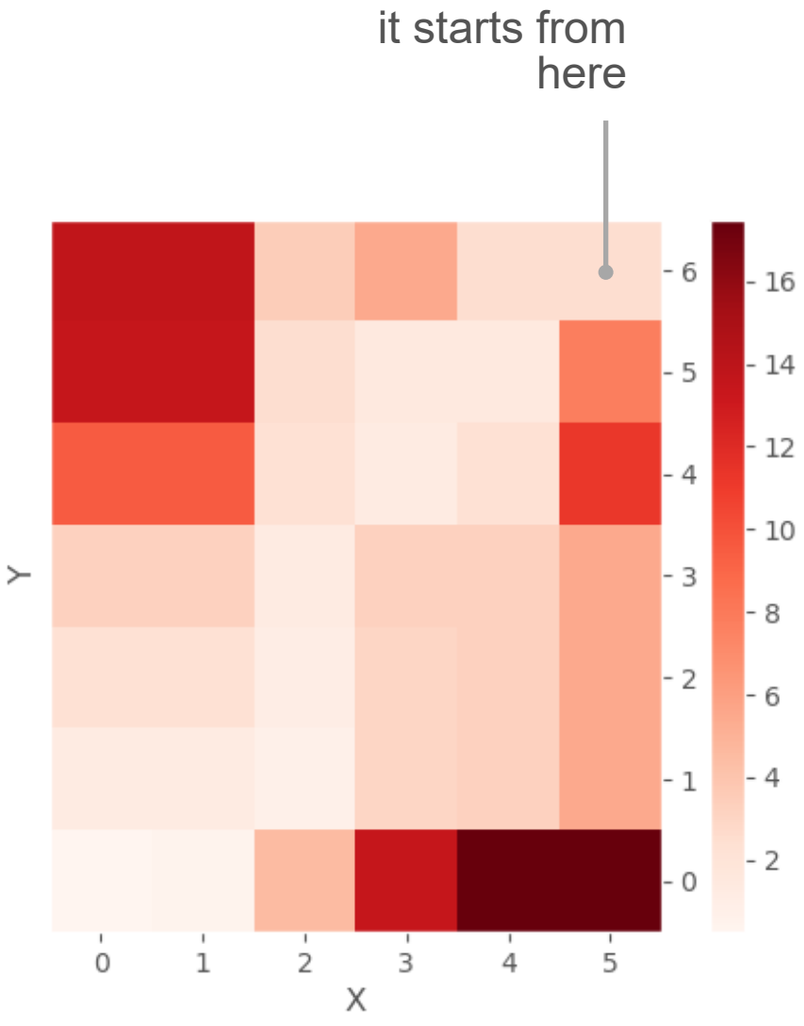
3. Search the optimal path

- Name the warping path as `path`.

```
> i       = len(y)-1
> j       = len(x)-1
> path    = [[j,i]]
> while (i > 0) and (j > 0):
      if i==0:
          j   = j-1
      elif j==0:
          i   = i-1
      else:
          if acuCost[i-1,j] == min(acuCost[i-1,j-1],
                                   acuCost[i-1,j],
                                   acuCost[i,j-1]):
              i   = i-1
          elif acuCost[i,j-1] == min(acuCost[i-1,j-1],
                                     acuCost[i-1,j],
                                     acuCost[i,j-1]):
              j   = j-1
          else:
              i   = i-1
              j   = j-1
      path.append([j,i])
> path.append([0,0])
```

a list; j stands for x axis, i stands for y axis

Wrapping process: Compare 3 neighboring squares and find the minimum cost square



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

3. Search the optimal path

- Create a function that plots the path

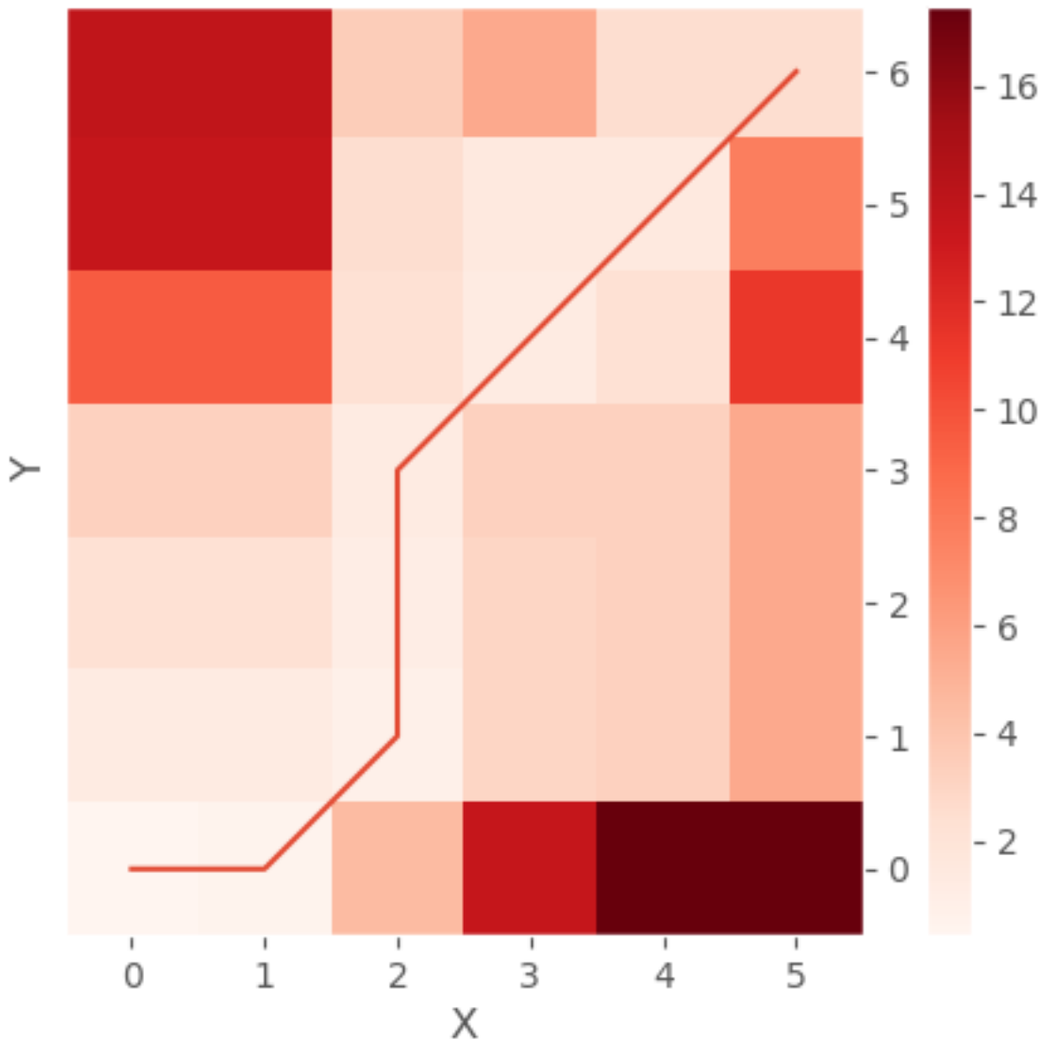
```
> def pltCostAndPath(acuCost,path,xlab="X",ylab="Y",clrmap="viridis"):
    px      = [pt[0] for pt in path]
    py      = [pt[1] for pt in path]

    imgplt  = pltDistances(acuCost,
                           xlab=xlab,
                           ylab=ylab,
                           clrmap=clrmap)

    plt.plot(px,py)

    return imgplt

> pltCostAndPath(acuCost,path,clrmap='Reds')
```



Now we want to plot the path; this code function helps us draw the path

© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

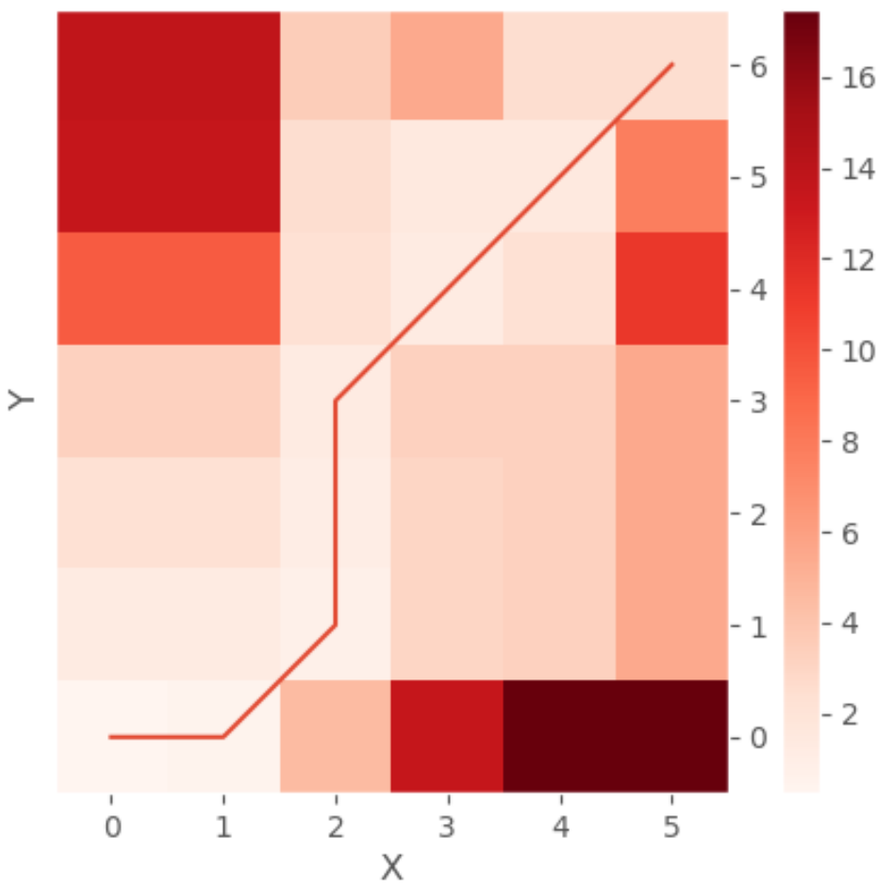
3. Search the optimal path

This cost parameter is also known as the wrapping cost

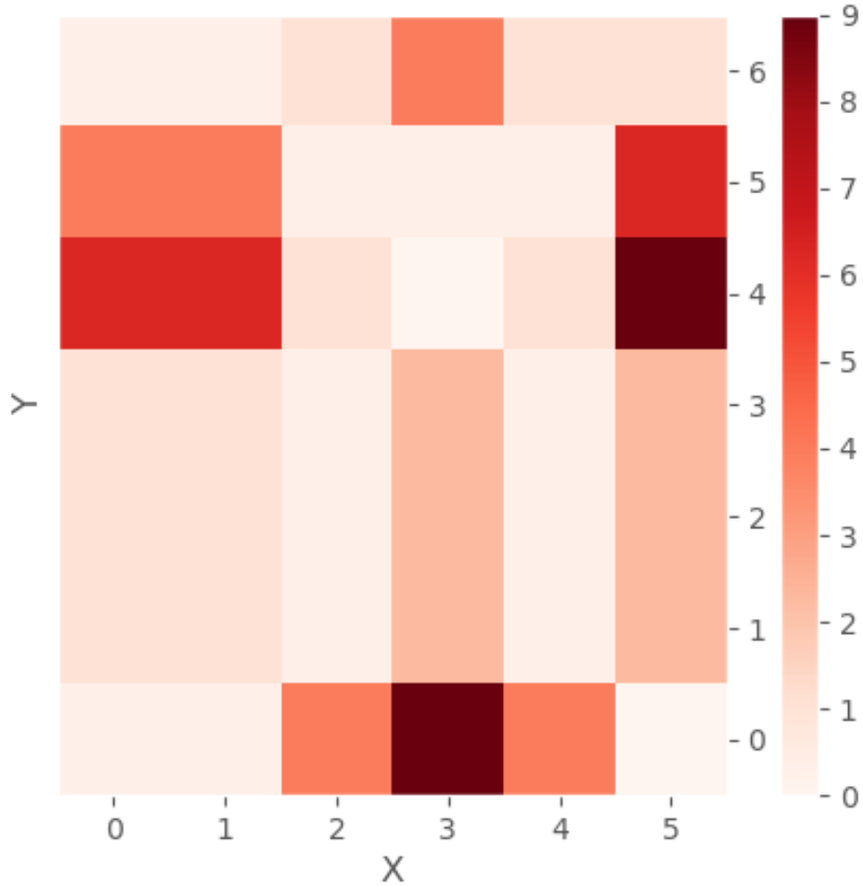
- Calculate the cost based on **`dists`**, which can be considered as a measure for similarity / distance

```
> cost         = 0
> for [j,i] in path:
      cost     = cost+dists[i,j]
> cost
: 2.5
```

acuCost



dists



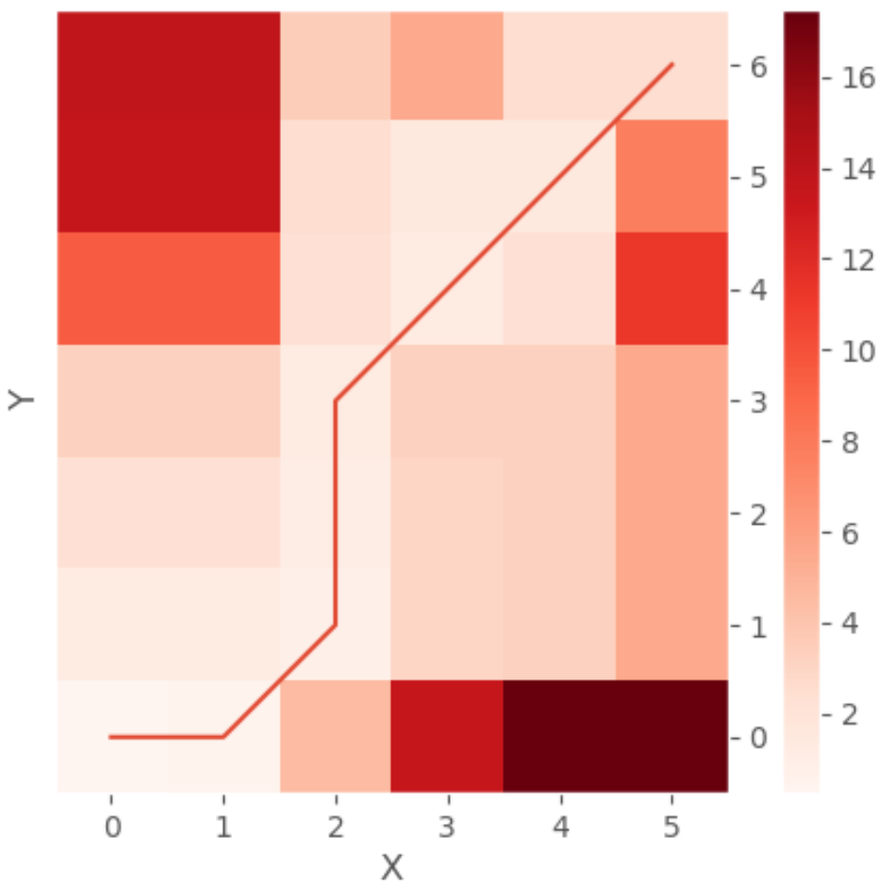
© 2024 National University of Singapore. All Rights Reserved.

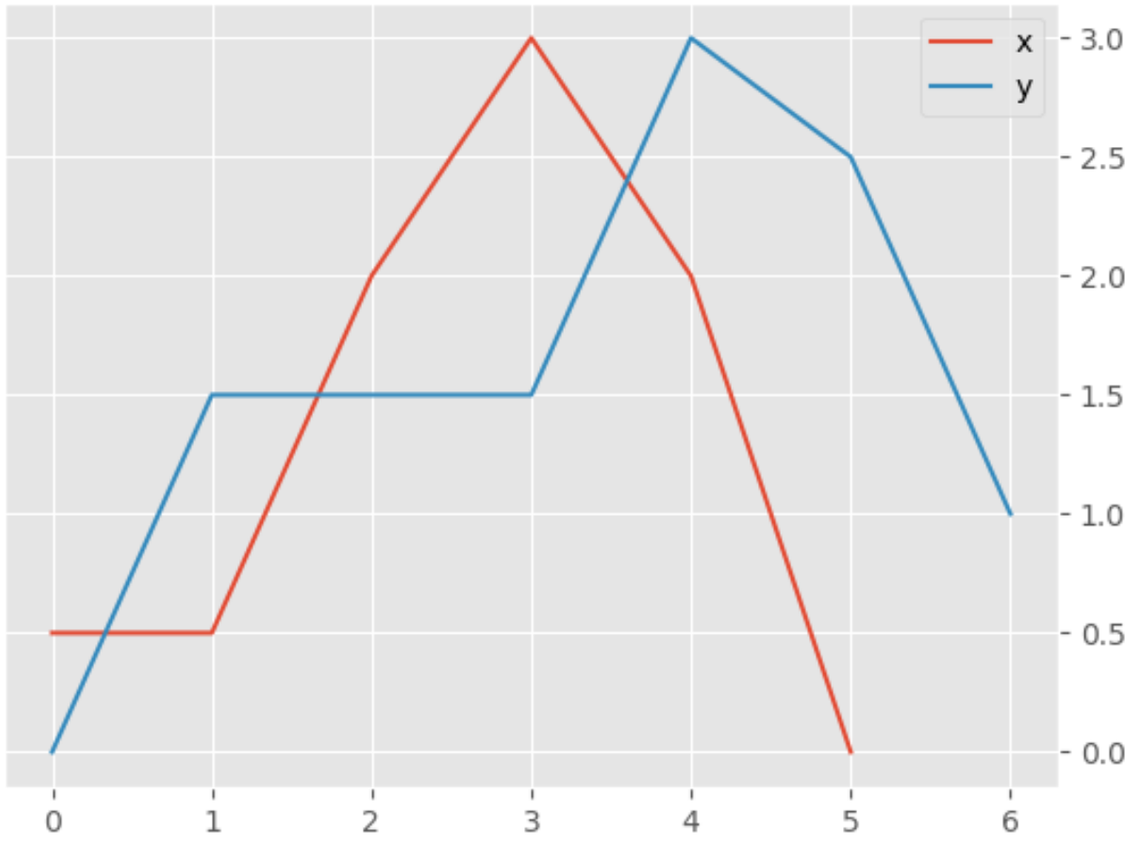
Dynamic Time Warping

3. Search the optimal path

- The implication

How to illustrate the mapping between the 2 signals (Right) based on the wrapping path in the Cost Matrix (Left)?





© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

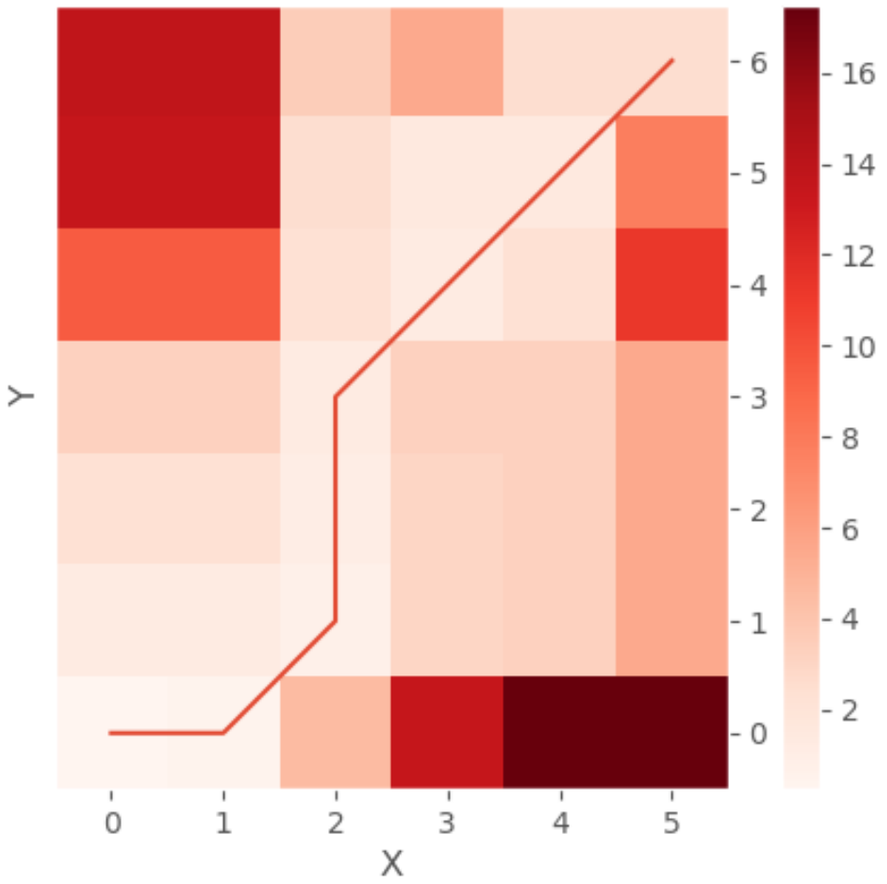
3. Search the optimal path

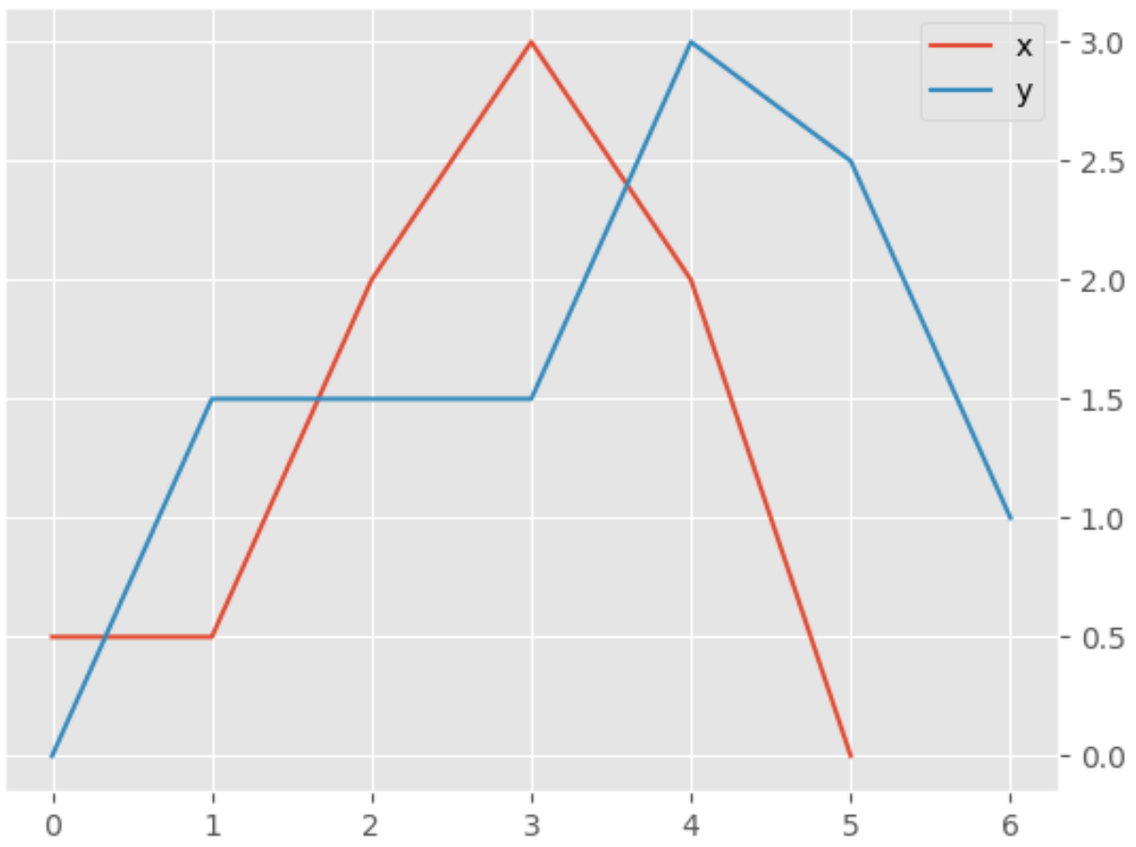
- The implication

How to plot a line that shows the correspondence between point 5 in X and point 6 in Y?

Ans: **plt.plot([5, 6], [X[5], Y[6]])**

`acuCost`





© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

3. Search the optimal path

- Plot the mapping of points between two signals

```
> def pltWarp(s1,s2,path,xlab="idx",ylab="Value"):
    imgplt      = plt.figure()

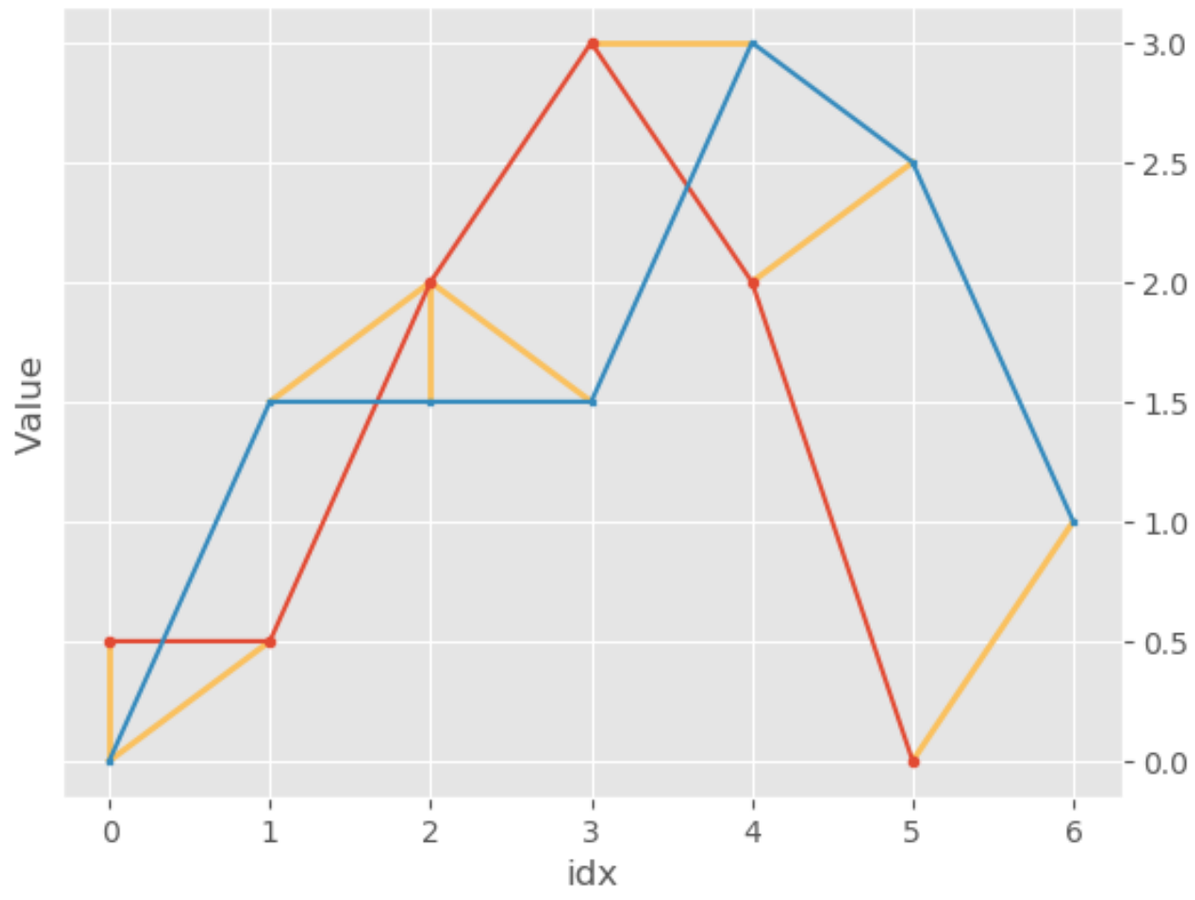
    for [idx1,idx2] in path:
        plt.plot([idx1,idx2],[s1[idx1],s2[idx2]],
                 color="C4",
                 linewidth=2)
    plt.plot(s1,
             'o-',
             color="C0",
             markersize=3)
    plt.plot(s2,
             's-',
             color="C1",
             markersize=2)
    plt.xlabel(xlab)
    plt.ylabel(ylab)

    return imgplt

> pltWarp(x,y,path)
```

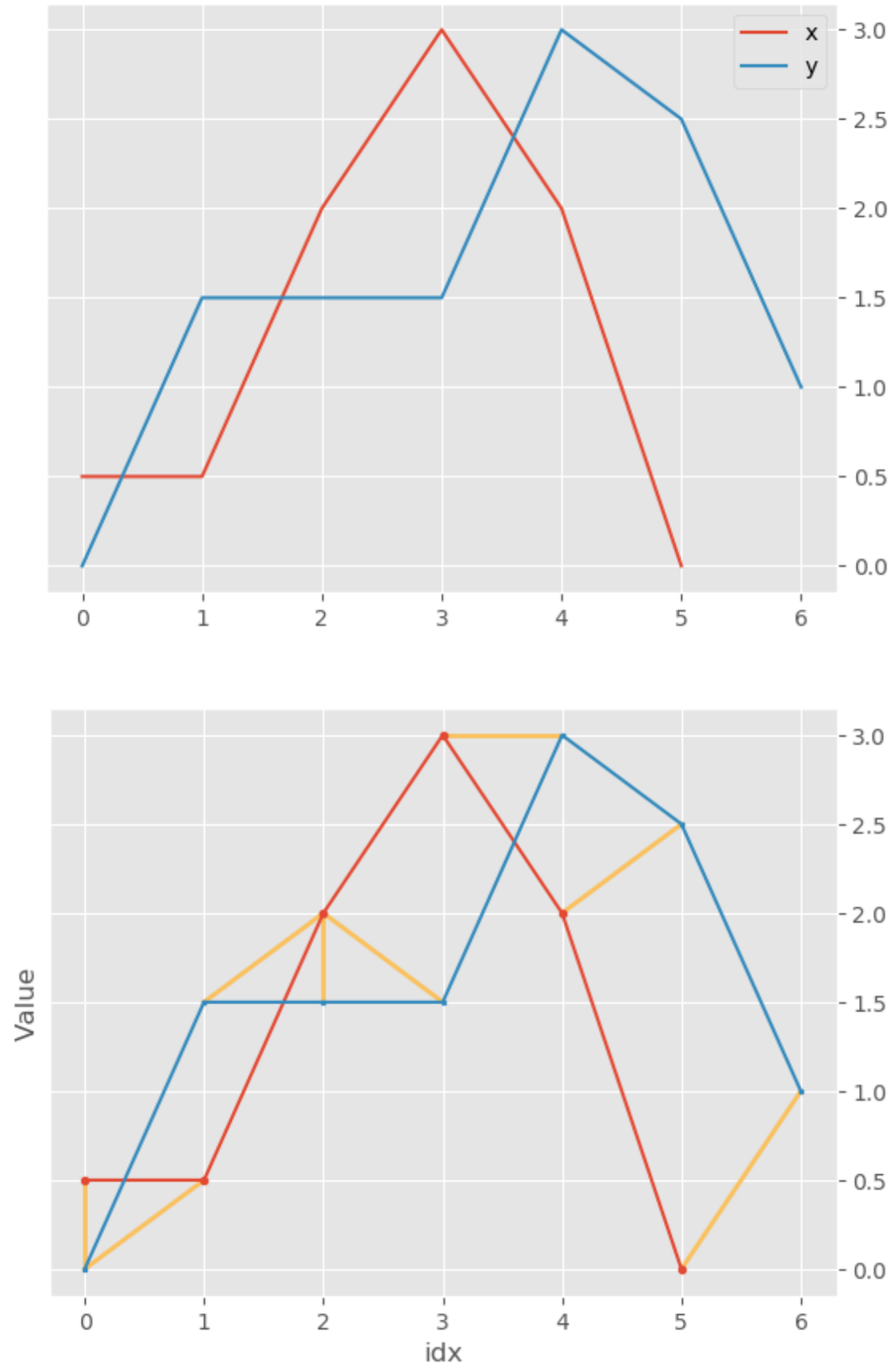
idx1 and idx2 are the interested time parameters

Plot the connections between s1 and s2 (yellow lines)



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping



© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Another example

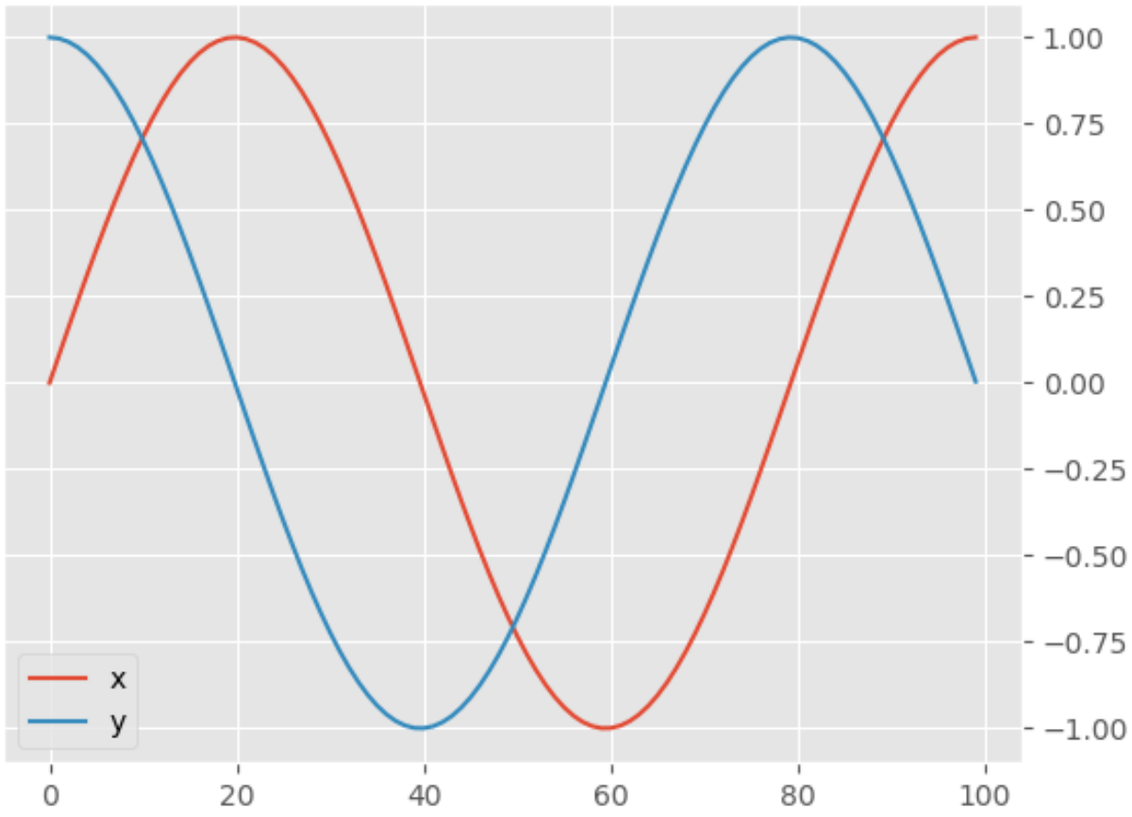
Another example with 2 different signals

- Define two signals as:

```
> x = np.sin(np.linspace(0,7.85,100))
> y = np.cos(np.linspace(0,7.85,100))
```

- Plot the two signals

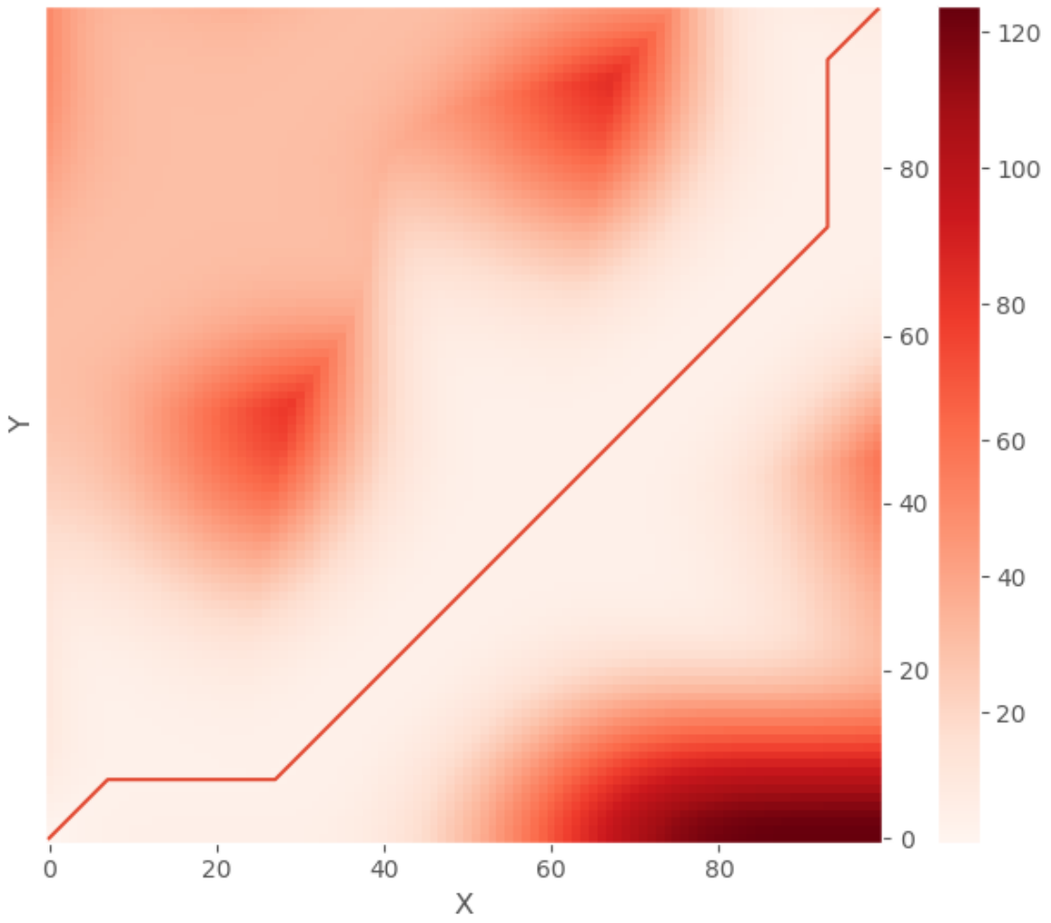
```
> plt.figure()
> plt.plot(x,
           color="C0",
           label='x')
> plt.plot(y,
           color="C1",
           label='y')
> plt.legend()
```

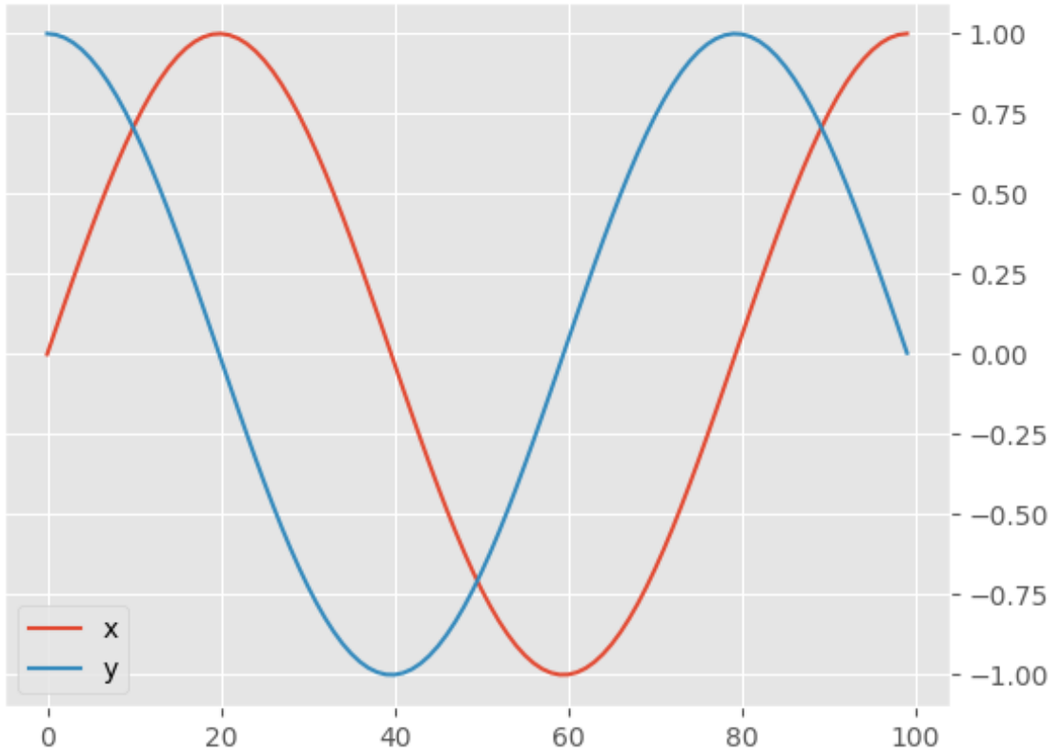


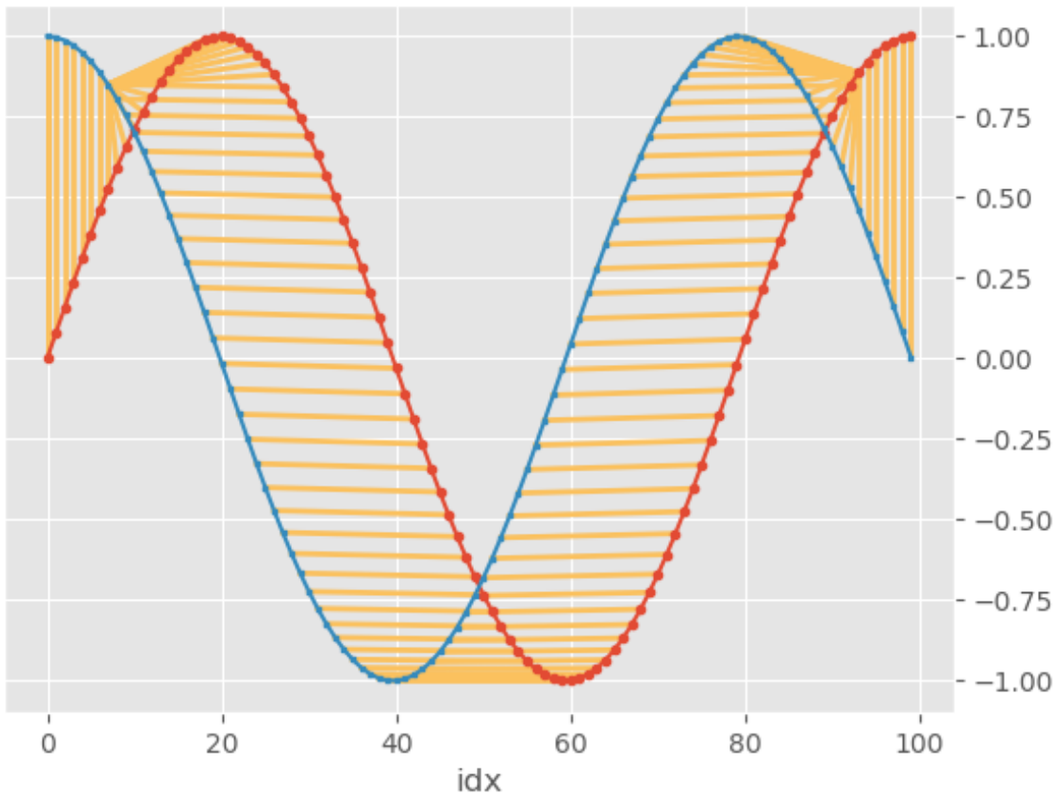
© 2024 National University of Singapore. All Rights Reserved.

Dynamic Time Warping

Another example







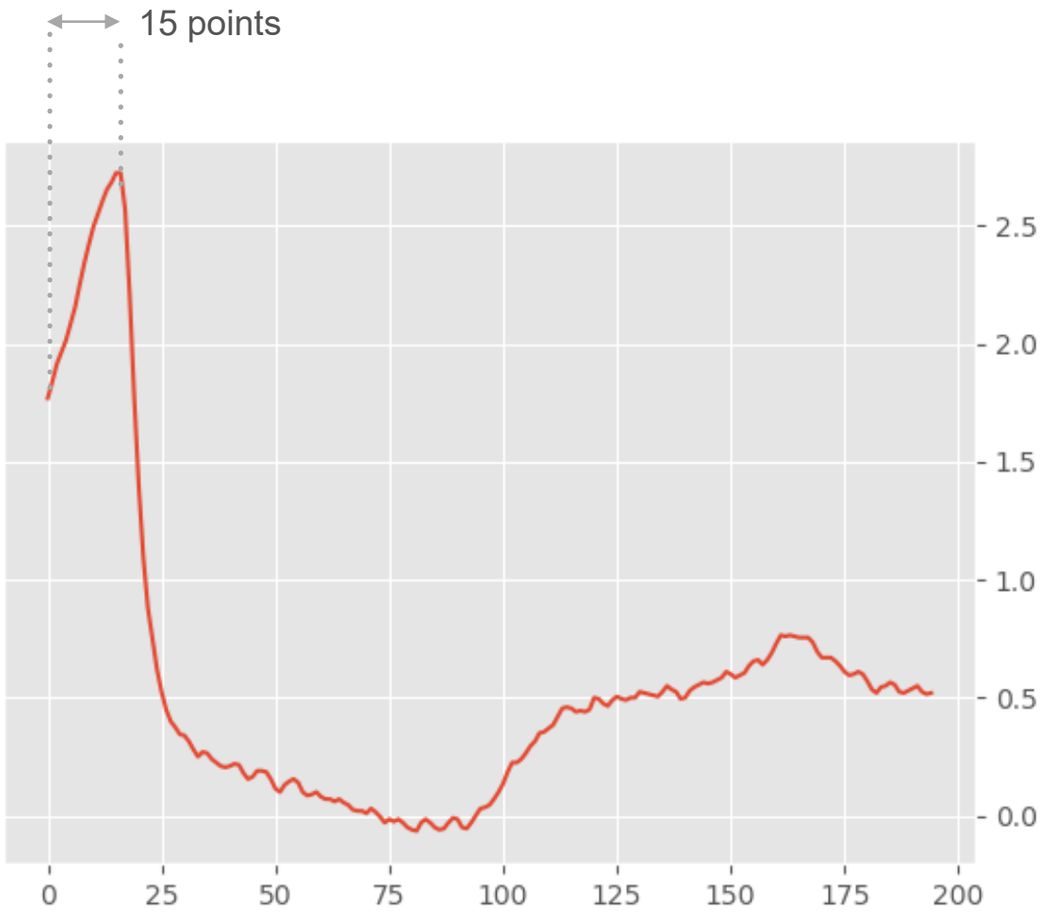
© 2024 National University of Singapore. All Rights Reserved.

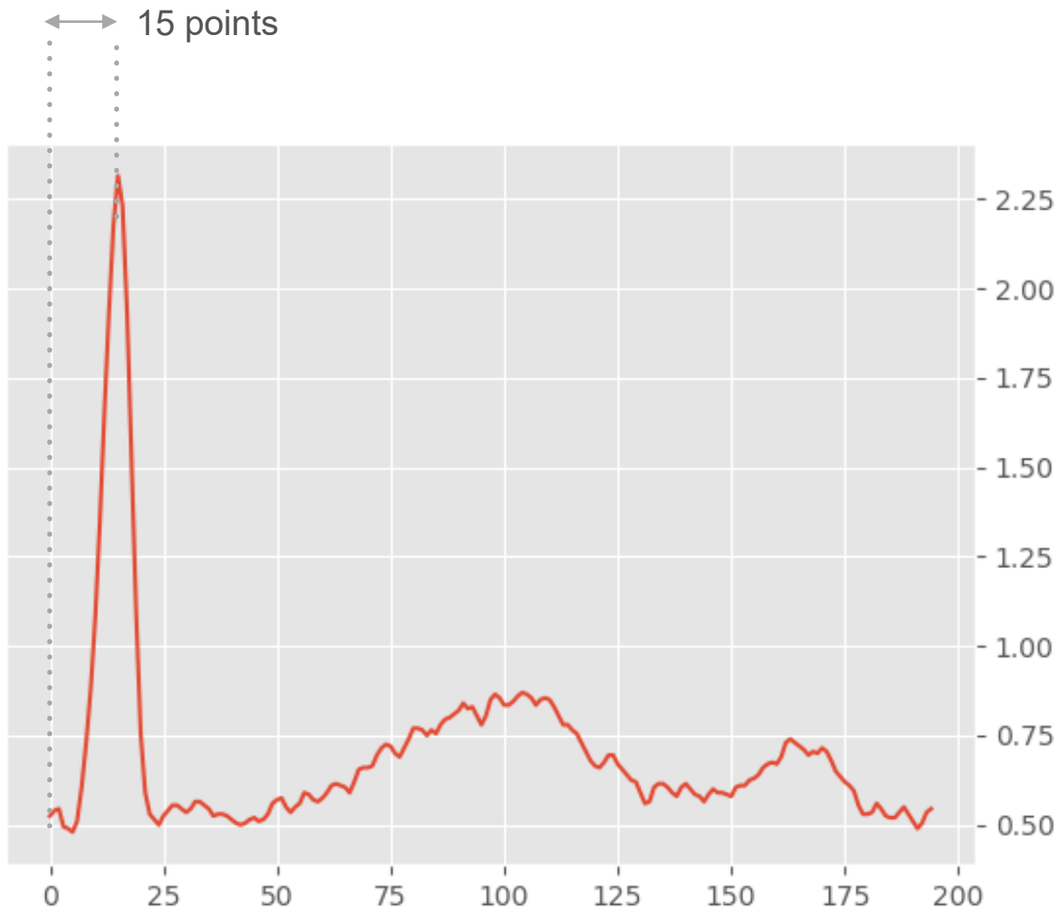
Back to the problem

How to start?

Use peak (i.e. **`findpeak`**) to estimate start of each ECG heartbeat by shifting 15 points to left from peak

- Before we compute similarity, must segment individual heartbeat signal
- With signal segmented, perform DTW





© 2024 National University of Singapore. All Rights Reserved.

Workshop

To start

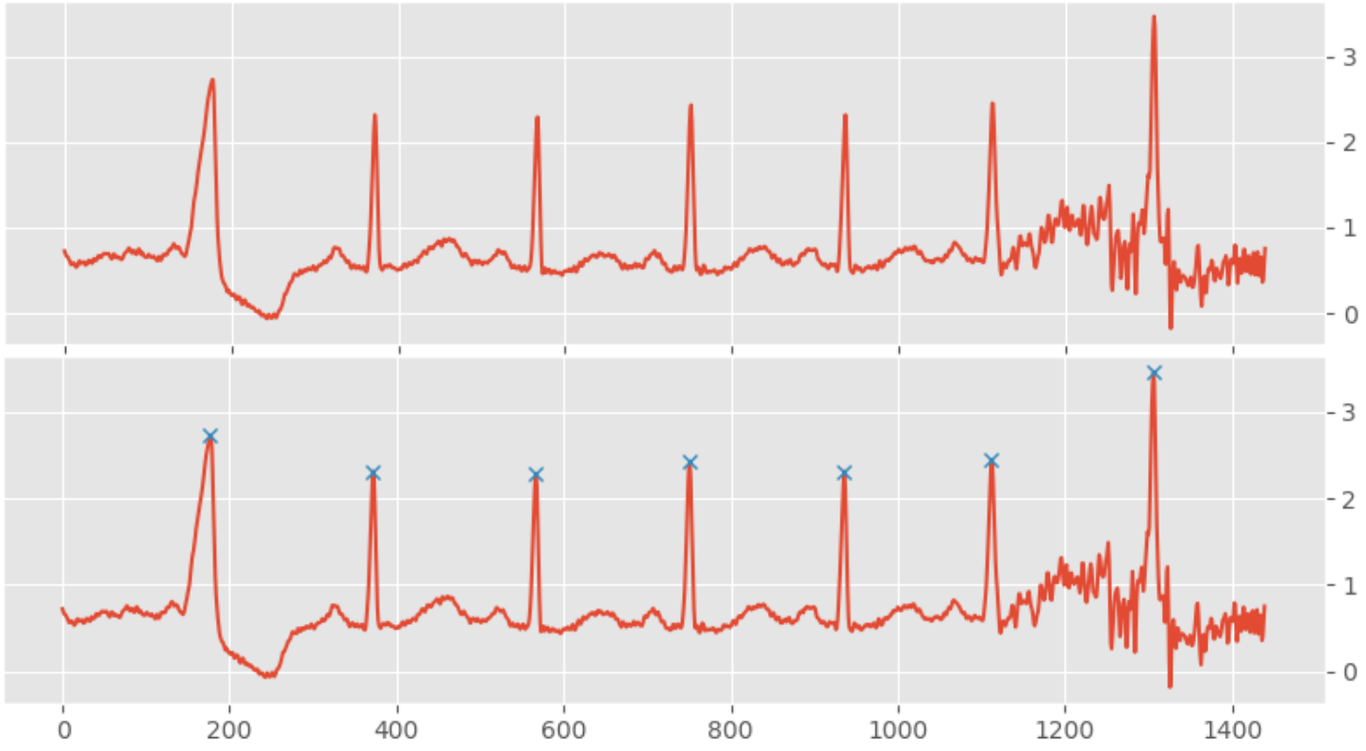
You need to complete a code function to chop (segment) each ECG heartbeat from the complete signal!

- Load the data

```
> l2D     = pd.read_csv('ecg2D.csv',
                        header=None)
> ECGs    = l2D[1].values
```

- Create a function with the below signature. The output is a list consists of all the ECG segments in a ECG signal

```
def extractECG(ecg,pks,offset=15):
```



General Procedures:

1. Load data
2. Perform peak detection
3. Extract segments based on the detected peaks

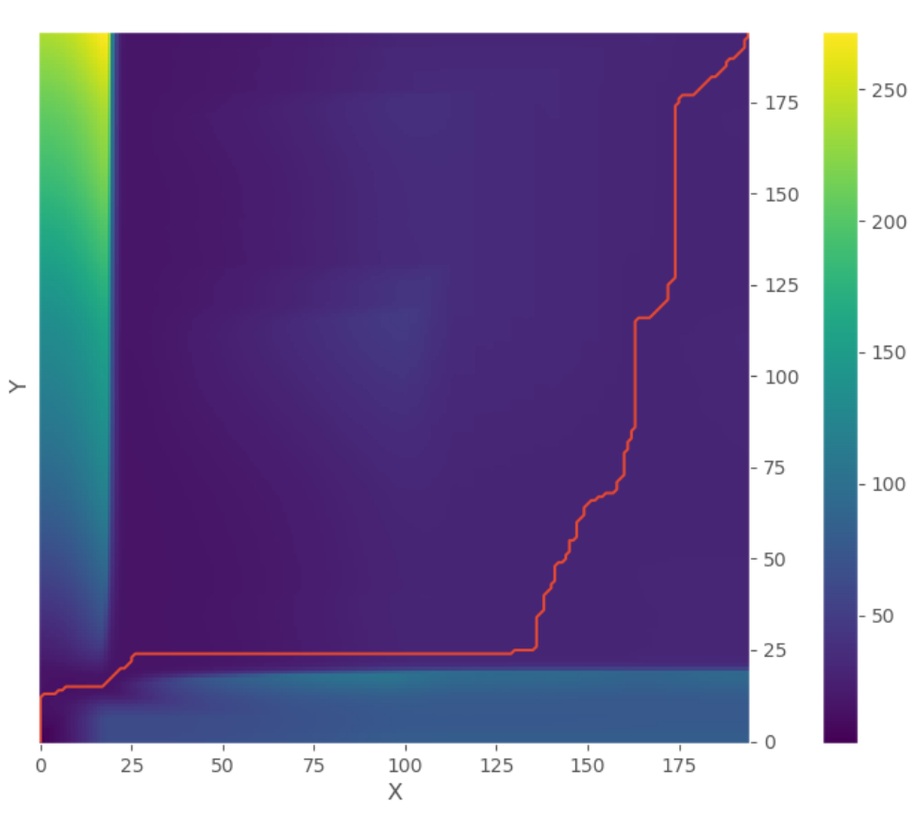
© 2024 National University of Singapore. All Rights Reserved.

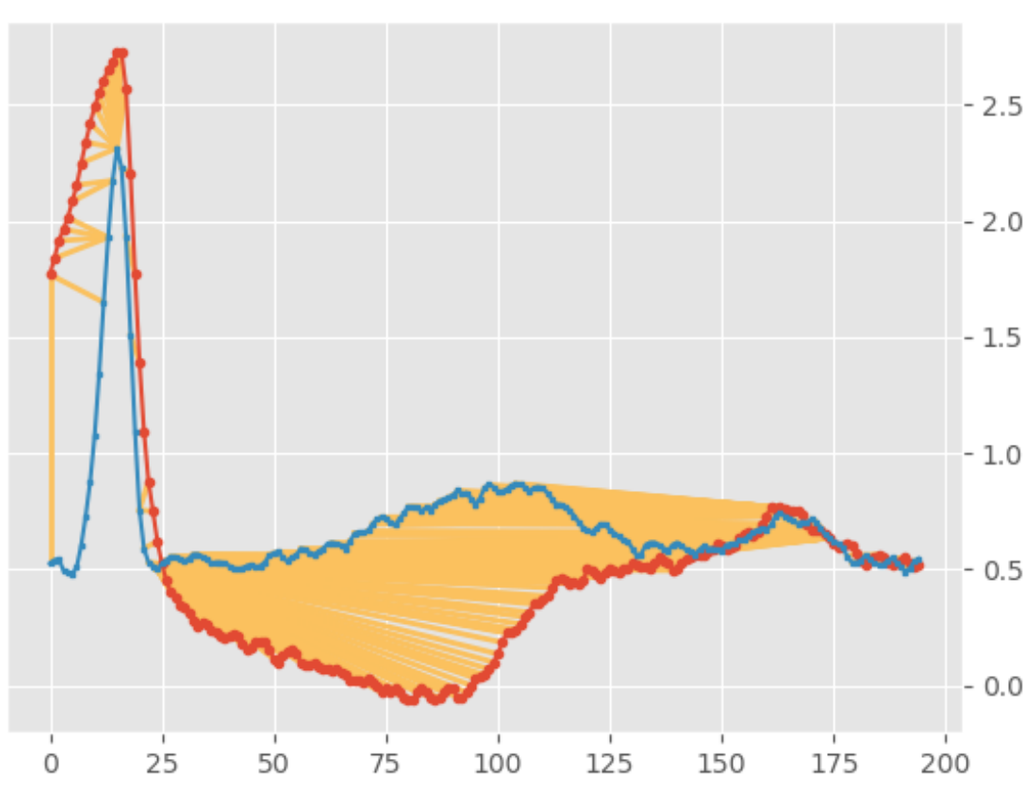
Workshop

Compute the accumulated costs, plot the optimal paths and warp

If you are done early, try on other signals
(i.e. other than the 2nd column of the `ecg2D` data)

- Make the comparisons between segment
  - 1 and 2
  - 2 and 3
  - 2 and 6





© 2024 National University of Singapore. All Rights Reserved.

Bonus Workshop [Do NOT need to submit this]

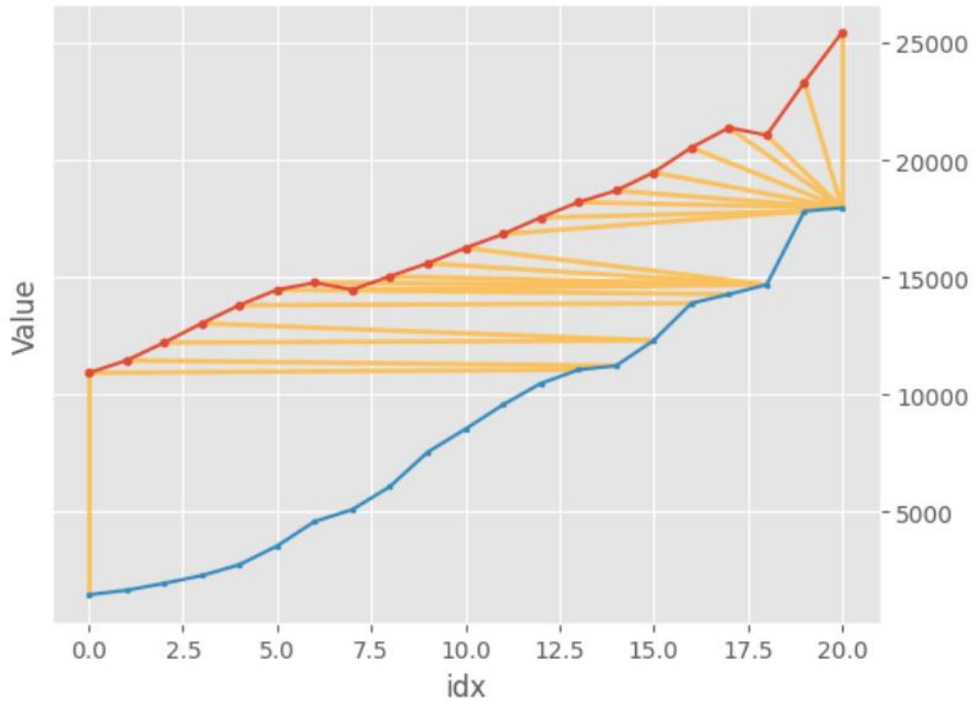
Comparing U.S., China & Japan GDP for the past 2 decades [2002 to 2022]

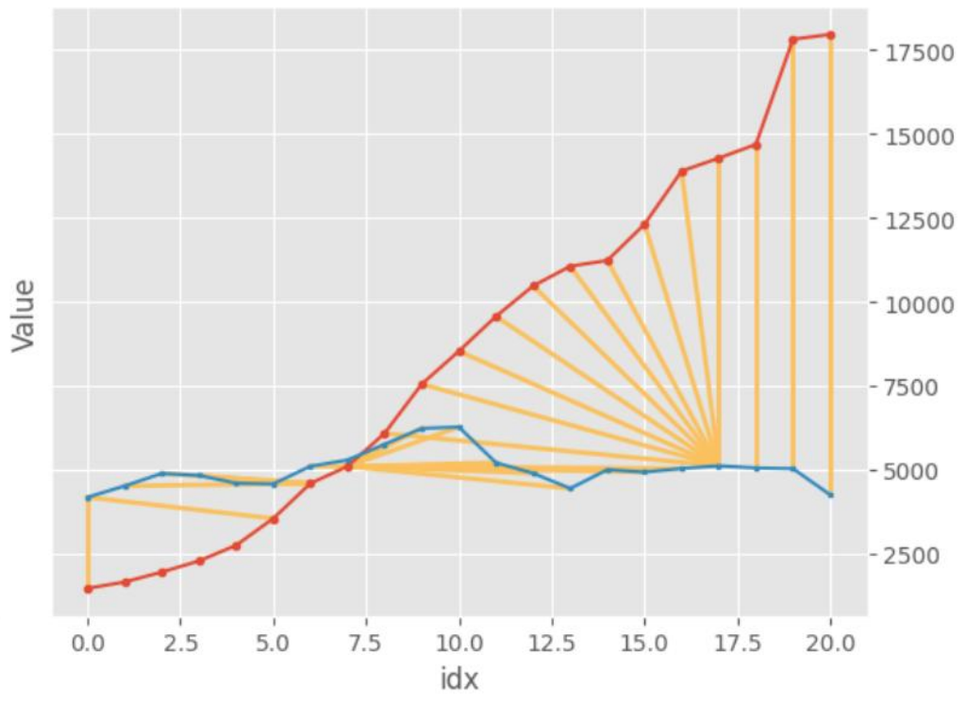
Using the GDP.csv data, use DTW to compare the following:

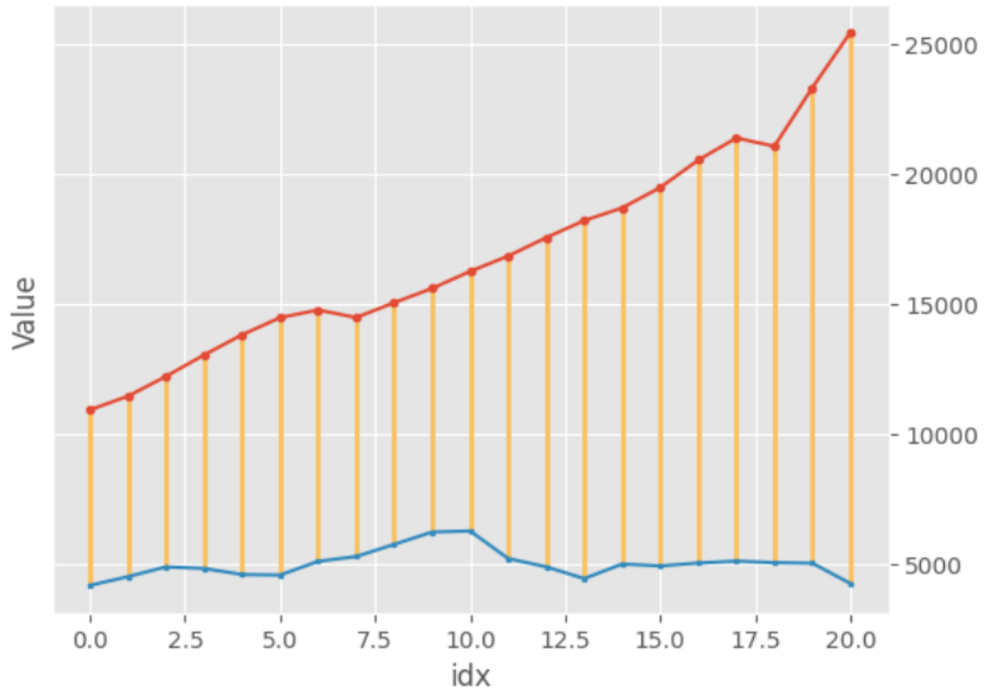
1. U.S GDP vs China GDP
2. China GDP vs Japan GDP
3. U.S GDP vs Japan GDP

The breakdown of the GDP.csv data file is as follows:

1. Column 1 – U.S GDP
2. Column 2 – China GDP
3. Column 3 – Japan GDP







© 2024 National University of Singapore. All Rights Reserved.

Appendix: Comparison among Various Methods

To measure similarity between signals

	Pros	Cons
Euclidean/Manhattan distance	Easy to implement; straightforward	Only works if the signals to be compared are of same length and preferably similar shape
DTW	Not necessary for signals to be of same length and shape; Computes faster than LCSS	Heavy computational burden (worse than Euclidean/Manhattan distance method); Unable to work if one of the signals is a partial type
Subsequence DTW (SS-DTW)	Can handle partial signals; Aligns a subsequence from one signal with another	Still computationally expensive compared to Euclidean/Manhattan; May still struggle with noisy signals
Longest common subsequence (LCSS)	Not necessary for signals to be of same length and shape; Allows for the use of partial and noisy signals	Heavy computational burden; Computes slower than DTW
Developed Longest Common Subsequence (DLCSS)	Not necessary for signals to be of same length and shape; Allows for the use of partial and noisy signals; Most accurate among the other methods	Heavy computational burden; Computes slower than DTW and LCSS

© 2024 National University of Singapore. All Rights Reserved.